区块链技术及应用技能赛

赛前培训资料

区块 1　区块 2　区块 3　区块 4　区块 5

参赛选手内部培训资料

2026 年安徽省区块链技术及应用技能赛

内 部 资 料

严禁外传 • 禁止复制 • 仅供参赛选手使用

内部资料

Contents

1 区块链零基础入门 5
- 1.1 计算机基础知识 . . . 5
  - 1.1.1 进制与编码 . . . 5
  - 1.1.2 网络基础 . . . 5
  - 1.1.3 数据库基础 . . . 5
  - 1.1.4 JSON 格式 . . . 6
- 1.2 职业道德与法律法规 . . . 6
  - 1.2.1 职业道德基本知识 . . . 6
  - 1.2.2 相关法律法规 . . . 6
- 1.3 通证与资产应用（补充） . . . 7
  - 1.3.1 什么是通证（Token）? . . . 7
  - 1.3.2 通证的分类 . . . 7

2 区块链核心概念 7
- 2.1 什么是区块链？ . . . 7
- 2.2 区块链 vs 传统数据库 . . . 8
- 2.3 区块链的类型 . . . 8

3 密码学基础 9
- 3.1 Hash 函数（哈希函数） . . . 9
  - 3.1.1 什么是 Hash？ . . . 9
  - 3.1.2 区块链中的 Hash 应用 . . . 9
  - 3.1.3 常用 Hash 算法 . . . 10
  - 3.1.4 Python 计算 Hash . . . 10
- 3.2 非对称加密（公钥加密） . . . 10
  - 3.2.1 核心概念 . . . 10
  - 3.2.2 椭圆曲线加密（ECC） . . . 13
- 3.3 数字签名 . . . 13
  - 3.3.1 签名过程 . . . 13
  - 3.3.2 区块链中的签名应用 . . . 14
- 3.4 Merkle Tree（默克尔树） . . . 14
  - 3.4.1 结构和原理 . . . 14
  - 3.4.2 Merkle Tree 的优势 . . . 14
- 3.5 密码学算法深度解析 . . . 15
  - 3.5.1 椭圆曲线密码学（ECC）数学原理 . . . 15
  - 3.5.2 SM2 签名算法的详细数学推导 . . . 16
  - 3.5.3 SHA-256 算法详细步骤 . . . 17

4 区块链底层数据结构 19
- 4.1 区块结构 . . . 19
  - 4.1.1 区块头（Block Header） . . . 19
  - 4.1.2 区块体（Block Body） . . . 20
- 4.2 交易结构（UTXO 模型） . . . 20
  - 4.2.1 什么是 UTXO？ . . . 20
  - 4.2.2 交易过程 . . . 20

内部资料

5 共识算法 21
5.1 拜占庭将军问题 . . . 21
5.2 PoW —工作量证明（Proof of Work） . . . 21
5.2.1 核心思想 . . . 21
5.2.2 PoW 的特点 . . . 22
5.3 PoS —权益证明（Proof of Stake） . . . 22
5.3.1 核心思想 . . . 22
5.3.2 PoS 的特点 . . . 22
5.4 DPoS —委托权益证明 . . . 23
5.4.1 核心思想 . . . 23
5.4.2 DPoS 的特点 . . . 23
5.5 PBFT —实用拜占庭容错 . . . 23
5.5.1 核心思想 . . . 23
5.5.2 三阶段协议 . . . 24
5.5.3 PBFT 的特点 . . . 24
5.6 Raft —非拜占庭容错共识 . . . 25
5.6.1 核心思想 . . . 25
5.6.2 三种角色 . . . 25
5.6.3 Raft vs PBFT . . . 25
5.7 共识算法深度解析 . . . 25
5.7.1 PoW 难度调整算法 . . . 25
5.7.2 PBFT 的数学证明 . . . 27
5.7.3 PoS 的罚没机制（Slashing） . . . 28
5.7.4 共识算法对比总结 . . . 29

6 智能合约 29
6.1 什么是智能合约？ . . . 29
6.1.1 生活化类比 . . . 30
6.2 以太坊和 EVM . . . 30
6.2.1 以太坊的诞生 . . . 30
6.2.2 EVM（以太坊虚拟机） . . . 30
6.2.3 Gas 机制 . . . 30
6.3 Solidity 合约示例 . . . 31

7 区块链的类型 31
7.1 公链（Public Blockchain） . . . 31
7.2 联盟链（Consortium Blockchain） . . . 32
7.3 私链（Private Blockchain） . . . 32

8 区块链应用领域 32
8.1 数字货币与金融应用 . . . 32
8.1.1 比特币（Bitcoin） . . . 32
8.1.2 稳定币（Stablecoin） . . . 33
8.1.3 央行数字货币（CBDC） . . . 33
8.2 资产数字化与供应链金融 . . . 33
8.2.1 资产数字化（Asset Tokenization） . . . 33
8.2.2 供应链金融 . . . 34
8.3 政务与存证 . . . 35

内部资料

8.3.1 电子政务 . . . 35
8.3.2 司法存证 . . . 35
8.4 数字身份 . . . 36

9 中国区块链发展与国密算法 36
9.1 国密算法概述 . . . 36
9.2 SM2 —非对称加密算法 . . . 36
9.2.1 什么是 SM2? . . . 36
9.2.2 SM2 的核心概念 . . . 36
9.2.3 SM2 的两大功能 . . . 37
9.3 SM3 —Hash 算法 . . . 38
9.3.1 什么是 SM3? . . . 38
9.3.2 SM3 vs SHA-256 . . . 38
9.3.3 SM3 的计算过程（简化） . . . 38
9.4 SM4 —对称加密算法 . . . 39
9.4.1 什么是 SM4? . . . 39
9.4.2 SM4 vs AES 对比 . . . 40
9.4.3 SM4 的加密过程 . . . 40
9.4.4 SM4 的工作模式 . . . 40
9.4.5 SM4 的实际应用 . . . 41
9.5 国密算法总结 . . . 42

10 数据结构基础 43
10.1 链表（Linked List） . . . 43
10.2 树（Tree） . . . 43
10.2.1 二叉树 . . . 43
10.2.2 Merkle Tree . . . 43
10.2.3 MPT（Merkle Patricia Tree） . . . 43
10.3 Merkle Tree 的数学性质 . . . 44
10.3.1 Merkle Tree 的高度与证明大小 . . . 44
10.3.2 Merkle Proof 的构造与验证 . . . 44
10.3.3 UTXO 模型的数学分析 . . . 45
10.3.4 比特币地址生成的完整流程 . . . 46
10.3.5 以太坊地址生成 . . . 47

11 链上身份（DID） 48
11.1 什么是 DID? . . . 48
11.1.1 DID 的格式 . . . 48
11.1.2 DID 的生成 . . . 48
11.1.3 DID 文档（DID Document） . . . 49
11.2 可验证凭证（VC） . . . 49

12 实操技能备考要点 50
12.1 实操技能要点 . . . 50
12.2 Python 编程要点 . . . 50
12.2.1 Hash 计算 . . . 50
12.2.2 Merkle Root 计算 . . . 51
12.2.3 简单区块链实现 . . . 51

内部资料

12.3 区块链浏览器实操 53
12.3.1 比特币浏览器使用 53
12.3.2 以太坊浏览器使用 53
12.4 数字签名实操 54
12.4.1 ECDSA 签名与验证（Python 实现） 54
12.4.2 SM2 签名与验证（国密算法） 55
12.5 备考建议 56

13 附录 A：核心术语速查表 57

14 附录 B：知识体系思维导图 58

15 附录 C：推荐学习资源 59

16 附录 D：实操环境搭建指南 59
16.1 Python 环境准备 59
16.2 区块链浏览器使用 59

17 附录 E：模拟试题 59
17.1 理论知识题 59
17.2 实操技能题 60
17.3 答案速查 61

18 算法实现详解 61
18.1 完整区块链实现 62
18.1.1 基础数据结构 62
18.2 密码学算法实现 66
18.2.1 完整 SM3 实现 66
18.3 课后练习与思考题 69
18.3.1 密码学练习 69
18.3.2 共识算法练习 70
18.3.3 编程练习 70
18.3.4 综合思考题 71

内部资料

1 区块链零基础入门

1.1 计算机基础知识

本节梳理最核心的计算机基础概念，可能以选择题或判断题形式出现。

1.1.1 进制与编码

- 二进制（**Bin**）：计算机内部只用 0 和 1 两种状态，比特币的 Hash 运算是二进制层面的计算
- 十六进制（**Hex**）：二进制太长，用十六进制表示更简洁。SHA-256 的输出就是 64 个十六进制字符
- 字符编码：UTF-8 是互联网上最通用的字符编码，中文在区块链系统中以 UTF-8 存储
- **Base58** 编码：比特币地址使用 Base58Check 编码，去掉了容易混淆的字符（0、O、l、I），更 human-readable

记住：SHA-256 输出 64 个十六进制字符 = 256 位二进制 = 32 字节

1.1.2 网络基础

- **IP** 地址：互联网上的”门牌号”，区块链节点通过 IP 互相通信
- **DNS**：域名系统，把人类可读的网址（如 google.com）转成 IP 地址
- **HTTP/HTTPS**：网页传输协议，区块链浏览器使用 HTTPS 保证安全
- **P2P** 网络：点对点网络，区块链的基础架构。没有中心服务器，所有节点平等

1.1.3 数据库基础

- 关系型数据库：MySQL、Oracle，用表格存储数据，有行和列
- 键值对数据库：Redis、LevelDB，用 Key-Value 存储，区块链常用
- 分布式数据库：数据分散存储在多台机器上，区块链是特殊的分布式数据库
- **ACID** 特性：原子性、一致性、隔离性、持久性。传统数据库追求 ACID，区块链追求最终一致性

内部资料

1.1.4 JSON 格式

区块链中大量数据以 JSON 格式传输和存储。

```
{
  "name": "Alice",
  "age": 25,
  "isStudent": false,
  "courses": ["Math", "CS", "Blockchain"]
}
```

重点提示：JSON 格式看起来像这样——“k̈ey: v̈alue, n̈umber: 123”。以太坊的智能合约调用和大多数区块链浏览器都返回 JSON 数据。

1.2 职业道德与法律法规

1.2.1 职业道德基本知识

- 诚实守信：区块链行业强调透明和可追溯，从业者必须诚实记录交易和数据
- 保护用户隐私：虽然区块链数据公开，但用户的真实身份需要保护，不能随意泄露
- 持续学习：区块链技术更新快，从业者需要不断学习新技术和新标准
- 安全意识：私钥管理、智能合约安全是区块链的核心，任何疏忽都可能导致巨大损失

1.2.2 相关法律法规

- 《中华人民共和国密码法》（2020 年 1 月 1 日实施）
  - 规范密码应用和管理，促进密码事业发展
  - 密码分为核心密码、普通密码和商用密码
  - 区块链中使用的 SM2/SM3/SM4 属于商用密码
- 《区块链信息服务管理规定》（2019 年 2 月 15 日实施）
  - 要求区块链信息服务提供者进行备案
  - 禁止利用区块链传播违法信息
  - 要求对使用者进行真实身份信息认证
- 《数据安全法》（2021 年 9 月 1 日实施）
  - 规范数据处理活动，保障数据安全
  - 重要数据需要分类分级保护
- 《个人信息保护法》（2021 年 11 月 1 日实施）
  - 规范个人信息处理活动
  - 区块链上存储个人信息需要符合法律规定

内部资料

重点提示：法律部分需要知道有哪些规定存在，以及规定的核心目的（如密码法是为了规范商业密码使用）。不需要背诵条文。

1.3 通证与资产应用（补充）

通证（Token）是区块链的独特概念。

1.3.1 什么是通证（Token）?

简单说：通证就是记录在区块链上的”权益凭证”。

通证的三个特征：

1. 可流通：可以在区块链上自由转让（如转让 NFT 所有权）
2. 可验证：任何人都可以在链上验证其真伪（查区块链即可）
3. 可编程：可以通过智能合约设定规则（自动分红、限量发行等）

类比：门票就是一种通证——有编号（可验证）、可转让（给他人）、有时效（可编程）。

1.3.2 通证的分类

- 同质化通证（**FT**，**Fungible Token**）：每个 Token 都一样，可以互换。类比人民币——每张 100 元价值相等。代表：比特币（BTC）、以太坊（ETH）
- 非同质化通证（**NFT**，**Non-Fungible Token**）：每个 Token 唯一，不可互换。类比房产证——每个房产证对应一套唯一的房子。代表：数字艺术品、游戏道具
- 证券化通证（**STO**）：代表真实资产的所有权（如房地产、股票），受证券法规管

重点：BTC/ETH = 同质化通证（FT），每个都一样；NFT = 非同质化通证，每个都唯一。

2 区块链核心概念

2.1 什么是区块链？

区块链（Blockchain）本质上是一个分布式数据库，但它和普通数据库有一个本质区别：数据一旦写入，就几乎不可能被修改或删除。

用更通俗的话说：区块链就是一个大家共同维护的、无法作弊的电子账本。

从记账场景理解

传统模式：你把钱存银行，银行在它的服务器上记账。你只能相信银行不会篡改你的记录。

区块链模式：所有参与者都保存一份完整的账本副本。任何人修改了自己的账本，其他人的账本都会拒绝匹配。就像全班同学都抄了一份课程表，有人偷偷改了自己的，老师一比对就知道谁在说谎。

区块链的核心特点：

1. 去中心化：没有单一的控制者，所有节点平等
2. 不可篡改：数据一旦写入，修改成本极高（需要控制超过 51% 的节点）
3. 可追溯：每一笔交易都有完整的历史记录，可以追踪资金流向
4. 透明性：所有交易对参与者公开（虽然身份可能是匿名的）

2.2 区块链 vs 传统数据库

特性	传统数据库	区块链
数据修改	管理员可以随时修改	几乎不可能修改
控制权	中心化（公司/机构控制）	去中心化（共识机制决定）
信任基础	相信中心机构	相信密码学和共识机制
查询速度	快（毫秒级）	慢（秒级甚至分钟级）
存储成本	低	高（每个节点都存一份）
适合场景	高频交易、大数据	价值转移、存证溯源

区块链不是万能的！它牺牲了效率换取了信任。如果你的场景不需要”去信任”，传统数据库可能更合适。

2.3 区块链的类型

- 公链（**Public Blockchain**）：任何人都可以参与，完全去中心化
  - 代表：比特币、以太坊
  - 特点：匿名、开放、抗审查
  - 共识：PoW、PoS
- 联盟链（**Consortium Blockchain**）：多个机构共同维护，部分去中心化
  - 代表：Hyperledger Fabric、R3 Corda、FISCO BCOS
  - 特点：需要许可、效率较高、适合企业
  - 共识：PBFT、Raft

- 私链（**Private Blockchain**）：单一机构控制，中心化
  - 代表：企业内部系统
  - 特点：完全可控、效率最高、去中心化程度低
  - 用途：内部审计、数据共享

中国主要发展联盟链，因为：1）符合监管要求；2）性能更高；3）适合企业级应用。比特币、以太坊是公链的代表。

3 密码学基础

密码学是区块链的基石。区块链中几乎所有的安全机制都依赖于密码学。本章内容占比很高，务必理解原理。

3.1 Hash 函数（哈希函数）

3.1.1 什么是 Hash？

Hash 函数是一个单向函数，可以把任意长度的输入转换成固定长度的输出。

```
输入："Hello World"
SHA-256输出：
    a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e

输入："Hello World!"（只多了一个感叹号）
SHA-256输出：7
    f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
```

Hash 函数的核心特性：

1. 单向性：知道输出，无法反推出输入（除非暴力破解）
2. 确定性：同样的输入，永远得到同样的输出
3. 雪崩效应：输入哪怕改变 1bit，输出完全不同
4. 抗碰撞：很难找到两个不同的输入，产生相同的输出

3.1.2 区块链中的 Hash 应用

1. 区块链接：每个区块存储前一个区块的 Hash，形成链式结构
2. 交易指纹：每笔交易计算 Hash 作为唯一标识

3. 数据完整性：任何数据篡改都会导致 Hash 变化，立即被发现
4. 工作量证明（**PoW**）：矿工通过计算 Hash 来竞争记账权

3.1.3 常用 Hash 算法

- **SHA-256**：比特币使用的算法，输出 256 位（64 个十六进制字符）
- **SM3**：中国国密标准，输出 256 位，与 SHA-256 安全强度相当
- **Keccak-256**：以太坊使用的算法（SHA-3 标准）
- **RIPEMD-160**：比特币地址生成中使用的算法，输出 160 位

重点：SHA-256 输出是 64 个十六进制字符（每字符 =4bit，$64 \times 4 = 256$bit）。SM3 是中国国密标准，与 SHA-256 安全强度相当。区块链中优先学习 SHA-256 和 SM3。

3.1.4 Python 计算 Hash

```python
import hashlib

# SHA-256计算
data = b"Hello Blockchain"
hash_result = hashlib.sha256(data).hexdigest()
print(f"SHA-256: {hash_result}")

# 验证雪崩效应
data1 = b"blockchain"
data2 = b"blockchaiN"  # 只改一个大写字母
h1 = hashlib.sha256(data1).hexdigest()
h2 = hashlib.sha256(data2).hexdigest()
print(f"原始: {h1}")
print(f"修改: {h2}")
print(f"不同字符数: {sum(c1 != c2 for c1, c2 in zip(h1, h2))}")
```

3.2 非对称加密（公钥加密）

3.2.1 核心概念

非对称加密使用一对密钥：

- 公钥（**Public Key**）：可以公开分享，用于加密和验证签名
- 私钥（**Private Key**）：必须保密，用于解密和创建签名

非对称加密的两个用途：

1. 加密通信：A 用 B 的公钥加密，只有 B 的私钥能解密
2. 数字签名：A 用私钥签名，任何人可以用 A 的公钥验证

内部资料

说明：虽然国密 SM2 用的”椭圆曲线”要在草稿纸上算极其复杂，但我们借用另一种最经典的公钥加密算法——RSA 算法，来举一个完美的”钟表数学（模运算）”例子。它的原理极其直观，只用小学水平的乘方和求余数就能演示全过程。
第一步：制造”公钥”和”私钥”（由系统在背后完成）
系统会通过几个简单的数学公式，给你生成一对配套的钥匙。

```
选定基数（钟表大小）：
系统挑选两个质数相乘，比如 3 × 11 = 33
我们的"数学钟表"一圈的大小就是 33

生成钥匙：
经过一套特定的数学推导（欧拉函数），系统为你配对出了两个数字：
- 公钥 e = 3
- 私钥 d = 7

现在的状态：
你向全世界公布："我的公钥是 3，我的钟表大小是 33。"
你自己偷偷藏好："我的私钥是 7。"
```

第二步：朋友用你的”公钥”给你发送机密（加密）
假设你的朋友想给你发送一个机密数字：4。
他拿到了你的公钥（3）和钟表大小（33），他只做一步运算：把机密数字做”公钥”次乘方，然后对 33 求余数。

```
计算公式：4^3 mod 33
计算过程：
4 × 4 × 4 = 64
64 ÷ 33 = 1 余 31

加密完成！
你的朋友把数字 31（密文）通过微信发给你。

黑客如果截获了 31，并且知道你的公钥是 3，
他也无法轻易倒推出原始数字是 4
（在数字极大的情况下，反向推导是数学界的超级难题）。
```

第三步：你用你的”私钥”还原机密（解密）
你收到了密文 31。接下来，见证奇迹的时刻到了。
你拿出你偷偷藏好的私钥（7）和钟表大小（33）。你做同样的动作：把密文做”私钥”次乘方，然后对 33 求余数。

```
计算公式：31^7 mod 33
计算过程：
31^7 = 27,512,614,111（这是一个很大的数）
27,512,614,111 ÷ 33 = 833,715,579 余 4

结果完美回到了 4！
```

这就是非对称加密的暴力美学：

```
你朋友用数字 3 把原文送进了一个复杂的数学迷宫变成了 31；
而你用数字 7，精准地把 31 从迷宫里拉了回来，变回了 4。

它们是一对天生的"数学互逆体"：
- 加密：原文^公钥 mod N = 密文
- 解密：密文^私钥 mod N = 原文
```

3.2.2 椭圆曲线加密（ECC）

区块链主要使用椭圆曲线加密（ECC），而不是 RSA。

- 优势：相同安全强度下，ECC 的密钥更短、计算更快
  - 256 位 ECC ≈ 3072 位 RSA 的安全强度
  - 比特币、以太坊使用 secp256k1 曲线
  - 中国国密 SM2 使用 sm2p256v1 曲线
- 比特币的密钥生成：
  1. 生成 256 位随机数作为私钥
  2. 使用 ECC 点乘法从私钥生成公钥
  3. 对公钥进行 Hash 运算生成地址

重点：256 位 ECC ≈ 3072 位 RSA 的安全强度。区块链（比特币、以太坊）使用 ECC（secp256k1 曲线）。中国使用 SM2（sm2p256v1 曲线）。公钥可以分享，私钥必须保密。

3.3 数字签名

数字签名是区块链身份认证的核心机制。

3.3.1 签名过程

1. 创建签名：用私钥对消息 Hash 进行”加密”
2. 验证签名：用公钥”解密”并比对 Hash

```
签名流程：
消息 -> Hash函数 -> Hash值 -> 私钥签名 -> 签名结果
验证流程：
签名结果 + 公钥 -> 解密出Hash值 -> 与消息的新Hash比对
```

重点：

- 签名 = 私钥加密（Hash 值）
- 验证 = 公钥解密（比对 Hash）
- 只有私钥持有者能创建有效签名
- 任何人都能验证签名真伪
- 签名不可伪造、不可抵赖

3.3.2 区块链中的签名应用

- 交易签名：证明交易确实由资产所有者发起
- 区块签名：证明区块确实由矿工/验证者生成
- 智能合约调用：证明调用者身份

3.4 Merkle Tree（默克尔树）

Merkle Tree 是区块链中组织交易数据的核心数据结构。

3.4.1 结构和原理

```
交易列表: [Tx1, Tx2, Tx3, Tx4]

        [Root Hash]          <- Merkle Root（存区块头）
         /       \
   [Hash01]     [Hash23]
    /    \\       /    \
[Tx1]  [Tx2]  [Tx3]  [Tx4]   <- 叶子节点（交易Hash）

计算过程:
Hash0 = Hash(Tx1)
Hash1 = Hash(Tx2)
Hash2 = Hash(Tx3)
Hash3 = Hash(Tx4)
Hash01 = Hash(Hash0 + Hash1)
Hash23 = Hash(Hash2 + Hash3)
Root = Hash(Hash01 + Hash23)
```

3.4.2 Merkle Tree 的优势

1. 快速验证：验证一笔交易是否存在，只需要 $O(\log n)$ 的 Hash 计算
2. 轻节点友好：SPV（简单支付验证）节点只需下载区块头（80 字节），不需要下载所有交易
3. 防篡改：任何一笔交易被修改，Merkle Root 就会改变，立即被发现

```
验证 Tx2 是否在区块中:
路径: Tx2 -> Hash(Tx3) -> Hash0 -> ... -> Merkle Root
如果最终计算得到的Merkle Root与区块头中的一致 -> 交易存在
```

重点：Merkle Root 存储在区块头中。如果交易列表中有任何一笔被修改，其 Merkle Root 就会改变。注意 Merkle Tree 是二叉树，节点数是偶数（奇数时复制最后一个节点）。

3.5 密码学算法深度解析

3.5.1 椭圆曲线密码学（ECC）数学原理

ECC 是现代区块链的数学基础。理解其原理有助于深入理解 SM2、比特币和以太坊的加密机制。

椭圆曲线的定义：
椭圆曲线是满足以下方程的点集：

$$y^2 = x^3 + ax + b \quad (4a^3 + 27b^2 \neq 0)$$

这个条件确保曲线没有奇点（自交或尖点），保证群运算的良定义性。

椭圆曲线上的群运算：
椭圆曲线上的点构成一个阿贝尔群，支持两种运算：

1. 点加法（$\mathbf{P + Q = R}$）：

```
几何解释：
1. 画一条直线通过P和Q两点
2. 这条直线与曲线相交于第三点R'
3. R是R'关于x轴的对称点

特殊情况：
- 如果P = Q（倍点）：使用切线代替割线
- 如果直线垂直（P = -Q）：结果为无穷远点O（单位元）
```

2. 标量乘法（$\mathbf{k * P}$）：

```
定义：k * P = P + P + ... + P（k次）

计算方法（二进制展开法/快速幂思想）：
例：计算 13 * P
13 = 1101_2
13 * P = 8P + 4P + P

步骤：
1. 计算 2P = P + P
2. 计算 4P = 2P + 2P
3. 计算 8P = 4P + 4P
4. 结果 = 8P + 4P + P

时间复杂度：O(log k) 次点加运算
```

椭圆曲线离散对数问题（**ECDLP**）：

核心难题（密码学安全性基础）：
给定：曲线 E、基点 G、点 Q = k * G 求：整数 k
这个计算在经典计算机上被认为是指数级困难的。当使用 256 位曲线时，暴力破解需要约 2^{128} 次运算，这在计算上不可行。
为什么安全？
正向计算（k * G）：使用快速幂算法，O(log k) 时间逆向计算（求 k）：没有已知的多项式时间算法

3.5.2 SM2 签名算法的详细数学推导

SM2 的签名过程涉及多个密码学原语，以下是完整的数学推导：

```
参数定义：
- 椭圆曲线E: y^2 = x^3 + ax + b (mod p)
- 基点G = (x_G, y_G)
- 曲线阶n（n是一个大素数）
- 私钥d: 随机整数, 1 <= d < n-1
- 公钥P = d * G = (x_P, y_P)
- 待签名消息M
```

签名生成过程：

1. 计算消息摘要：
$$e = \mathrm{SM3}(\mathrm{Z}\|M)$$
其中 Z 是用户标识和公钥的 Hash，|| 表示连接
2. 生成随机数：选择随机数 k，满足 $1 \le k < n-1$
3. 计算椭圆曲线点：
$$(x_1, y_1) = k \cdot G$$
4. 计算 **r**：
$$r = (e + x_1) \bmod n$$
如果 $r = 0$ 或 $r + k = n$，重新选择 k
5. 计算 **s**：
$$s = ((1+d)^{-1} \cdot (k - r \cdot d)) \bmod n$$
如果 $s = 0$，重新选择 k
6. 签名结果：(r, s)

签名验证过程：

1. 验证 $1 \le r < n$ 和 $1 \le s < n$
2. 计算 $e' = \mathrm{SM3}(\mathrm{Z}\|M')$
3. 计算 $t = (r + s) \bmod n$，若 $t = 0$ 则验证失败

4. 计算椭圆曲线点：$(x_1', y_1') = s \cdot G + t \cdot P$
5. 计算 $R = (e' + x_1') \bmod n$
6. 验证 $R \stackrel{?}{=} r$，若相等则签名有效

为什么验证公式有效？
我们需要证明：如果签名正确生成，则验证必然通过。
推导：

$$\begin{aligned}(x_1', y_1') &= s \cdot G + t \cdot P \\ &= s \cdot G + (r + s) \cdot d \cdot G \\ &= (s + rd + sd) \cdot G \\ &= (s(1 + d) + rd) \cdot G\end{aligned}$$

由签名公式：$s = (1 + d)^{-1} \cdot (k - rd) \pmod n$
两边乘以 $(1 + d)$：

$$s(1 + d) = k - rd \pmod n$$

因此：

$$s(1 + d) + rd = k \pmod n$$

代回：

$$(x_1', y_1') = k \cdot G = (x_1, y_1)$$

所以：

$$R = (e' + x_1') \bmod n = (e + x_1) \bmod n = r$$

证毕。□

3.5.3 SHA-256 算法详细步骤

SHA-256 是比特币使用的 Hash 算法，理解其内部结构有助于理解挖矿过程。

```
SHA-256参数：
- 输出长度：256位（32字节）
- 块大小：512位（64字节）
- 轮数：64轮
- 字大小：32位
```

预处理步骤：

1. 消息填充：

```
步骤：
1. 在消息末尾添加一个'1'位
2. 添加k个'0'位，使得 (消息长度 + 1 + k) ≡ 448 (mod 512)
3. 最后添加64位的原始消息长度（大端序）

示例：对消息"abc"（24位）进行填充
```

```
原始：01100001 01100010 01100011（24位）
步骤1：01100001 01100010 01100011 1（25位）
步骤2：添加423个0，使总长度达到448位
步骤3：添加64位长度值（0x0000000000000018）最终块：512位
```

2. 初始化 **Hash** 值（前 8 个素数的平方根的小数部分）：

```
H0 = 0x6a09e667
H1 = 0xbb67ae85
H2 = 0x3c6ef372
H3 = 0xa54ff53a
H4 = 0x510e527f
H5 = 0x9b05688c
H6 = 0x1f83d9ab
H7 = 0x5be0cd19
```

主循环（每 **512** 位块）：

```
对每一块进行64轮压缩：

1. 消息扩展：
   - 将512位块分成16个32位字：W[0..15]
   - 扩展为64个字：
     for t = 16 to 63:
       W[t] = σ1(W[t-2]) + W[t-7] + σ0(W[t-15]) + W[t-16]

2. 初始化工作变量：
   a = H0, b = H1, c = H2, d = H3
   e = H4, f = H5, g = H6, h = H7

3. 64轮压缩（t = 0 to 63）：
   T1 = h + Σ1(e) + Ch(e,f,g) + K[t] + W[t]
   T2 = Σ0(a) + Maj(a,b,c)
   h = g
   g = f
   f = e
   e = d + T1
   d = c
   c = b
   b = a
   a = T1 + T2

4. 更新Hash值：
   H0 += a, H1 += b, ..., H7 += h
```

辅助函数定义：

$$
\begin{aligned}
\mathrm{Ch}(x,y,z) &= (x \wedge y) \oplus (\neg x \wedge z) \\
\mathrm{Maj}(x,y,z) &= (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \\
\Sigma_0(x) &= \mathrm{ROTR}^2(x) \oplus \mathrm{ROTR}^{13}(x) \oplus \mathrm{ROTR}^{22}(x) \\
\Sigma_1(x) &= \mathrm{ROTR}^6(x) \oplus \mathrm{ROTR}^{11}(x) \oplus \mathrm{ROTR}^{25}(x) \\
\sigma_0(x) &= \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x) \\
\sigma_1(x) &= \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x)
\end{aligned}
$$

其中 ROTR 表示循环右移，SHR 表示逻辑右移。

SHA-256 在 PoW 中的应用：
比特币挖矿就是不断修改 Nonce，计算区块头的 SHA-256d（两次 SHA-256），直到结果小于目标值。

```
挖矿伪代码：
while True:
    block_header.nonce += 1
    hash1 = SHA-256(block_header)
    hash2 = SHA-256(hash1)
    if hash2 < target:
        return block_header.nonce  # 找到有效区块！
```

4 区块链底层数据结构

4.1 区块结构

一个区块由两部分组成：区块头（**Block Header**）和区块体（**Block Body**）。

4.1.1 区块头（Block Header）

固定 80 字节，包含 6 个字段：

字段	长度	说明
Version	4 字节	区块版本号
Previous Block Hash	32 字节	前一个区块的 Hash（链式结构）
Merkle Root	32 字节	所有交易的 Merkle Tree 根 Hash
Timestamp	4 字节	区块创建时间（Unix 时间戳）
Bits	4 字节	难度目标（编码后的目标值）
Nonce	4 字节	随机数（PoW 挖矿用）

重点：区块头总共 80 字节，是 PoW 工作量证明的直接操作对象。父区块哈希将区块串成链。Nonce 是 4 字节无符号整数，意味着在 4 字节范围内（大约 43 亿次）找不到有效 Hash 时，需要其他策略（如改变 Coinbase 交易中的 ExtraNonce）。

4.1.2 区块体（Block Body）

包含交易列表，大小不固定：

- 交易数量（变长整数）
- 交易列表（每笔交易 200-500 字节不等）

  比特币区块大小限制：1MB（隔离见证后 4MB）

4.2 交易结构（UTXO 模型）

比特币使用 **UTXO（Unspent Transaction Output）**模型，不是账户余额模型。

4.2.1 什么是 UTXO？

UTXO = 未花费的交易输出。可以理解为" 区块链上的零钱"。

```
类比现实：比特币的"余额"就像你钱包里的现金。
  - 你有：1张100元、2张50元、5张10元
  - 钱包余额 = 100 + 50×2 + 10×5 = 250元

比特币也一样：
  - UTXO #A: 0.5 BTC（来自某笔交易）
  - UTXO #B: 1.2 BTC（来自另一笔交易）
钱包余额 = 1.7 BTC
```

4.2.2 交易过程

```
小明要转1 BTC给小红：

小明的UTXO：
  - UTXO #1: 0.6 BTC
  - UTXO #2: 0.8 BTC
  - 总计：1.4 BTC

交易创建：
  输入：引用 UTXO #1 和 UTXO #2（作为花费）
  输出：
  - 新建 UTXO #A: 1.0 BTC（归属小红）
  - 新建 UTXO #B: 0.399 BTC（归属小明，找零）
  - 手续费：0.001 BTC（给矿工）

结果：
```

```
  - UTXO #1 和 #2 被标记为"已花费"
  - UTXO #A 和 #B 成为新的"未花费输出"
```

重点：UTXO 模型的核心是”输入引用过去的输出，产生新的输出”。理解 UTXO vs 账户余额模型的区别：银行是”记账”（A 账户减，B 账户加），比特币是”记录每笔零钱的归属”（UTXO 的流转）。以太坊使用的是账户余额模型。

5 共识算法

共识算法解决的是分布式系统中如何达成一致的问题。

5.1 拜占庭将军问题

由 Leslie Lamport 等人于 1982 年提出，是区块链共识机制最重要的理论来源。

```
情境：10位拜占庭将军分别带领一支军队包围了一座敌人城市。

问题：
- 将军们只能通过信使传递消息（没有电话、没有网络）
- 其中一些将军可能是叛徒，会发送错误信息
- 忠诚的将军们需要就"进攻还是撤退"达成一致

问题：如何让所有忠诚的将军就"进攻还是撤退"达成一致的行动计划？
```

将将军视为区块链节点，信使视为网络消息，叛徒视为恶意节点。

拜占庭容错（**BFT**）：系统能容忍最多 f 个恶意节点，仍能达成一致。

- $n \geq 3f + 1$：系统能容忍最多 f 个恶意节点（即恶意节点不超过总数的 1/3）
- 10 个将军，最多容忍 3 个叛徒
- 100 个节点，最多容忍 33 个恶意节点

5.2 PoW —工作量证明（Proof of Work）

5.2.1 核心思想

谁算力强，谁有话语权。通过消耗计算资源（算力）来竞争记账权。

矿工不断改变一个叫 Nonce 的随机数，对区块头反复计算 Hash，直到 Hash 值小于某个目标值。

```
目标：Hash(区块头 + Nonce) $<$\texttt{0000FFFFFFFFFFFFFFFF}\\ \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFF}

```

```
矿工尝试：
Nonce=0     -> Hash=8f8e2a1b3c4d5e6f...
Nonce=1     -> Hash=3a4b5c6d7e8f9a0b...Nonce=...（数十亿次尝试）
Nonce=12345678 -> Hash=00001a2b3c4d5e6f...  □ 找到啦！
```

5.2.2 PoW 的特点

- 优点：
  - 安全性高：攻击成本极高（需要控制 51% 算力）
  - 去中心化程度高：任何人都可以参与挖矿
  - 公平性：算力即权力，规则透明
- 缺点：
  - 能耗巨大：比特币年耗电量超过阿根廷全国
  - 效率低：每秒只能处理 3-7 笔交易
  - 算力集中：矿池控制大部分算力，有中心化趋势

重点：PoW 的核心是" 算力竞争，谁先找到有效 Hash 谁赢"。难度由 Bits 字段编码。51% 攻击是 PoW 的最大威胁，但实际攻击成本远超收益。比特币使用 SHA-256d（两次 SHA-256）作为 PoW 函数。

5.3 PoS —权益证明（Proof of Stake）

5.3.1 核心思想

根据持有的代币数量和时间来选择区块验证者。持有的越多、被锁定的时间越长，被选中出块的概率越高。类似于" 股东大会投票"，股份越多，话语权越大。

```
PoS选择公式（简化）：
选择概率 $\propto$ 质押代币数量 × 质押时间

示例：
  - 持有1000 ETH，锁定1年 -> 权重 = 1000 × 1 = 1000
  - 持有500 ETH，锁定2年 -> 权重 = 500 × 2 = 1000
  - 权重相同，随机选一个
```

以太坊 2.0 的 PoS 使用随机化（**Randomized**）方法——RANDAO + VDF（可验证延迟函数）产生随机数，结合质押权重选择出块者。

5.3.2 PoS 的特点

- 优点：
  - 能耗极低：比 PoW 节能 99.95%
  - 效率高：出块速度快，每秒可处理数千笔交易

  - 攻击成本高：攻击者需要购买并质押大量代币，攻击后代币会贬值

- 缺点：
  - 富者愈富：持币多的人更容易获得奖励
  - 无利害关系（Nothing at Stake）：验证者可能在多个分叉上同时签名
  - 初始分配问题：代币初始分配可能不公平

重点：PoS 中”质押”（Stake）意味着将代币作为保证金。如果验证者作恶，其质押的代币会被罚没（Slashing）。以太坊 PoS 最多可罚没验证者 32 ETH。这比 PoW 的 51% 攻击更经济高效，能耗降低约 99.95%。

5.4 DPoS —委托权益证明

5.4.1 核心思想

代币持有者投票选举出少数节点（如 21 个）负责出块和验证。类似于”代议制民主”，大家投票选代表。

```
EOS的DPoS机制：
1. EOS持有者投票选出21个"区块生产者"（BP）
2. 21个BP轮流出块（每3秒一个区块）
3. BP获得区块奖励，需要回馈社区
4. 如果BP表现不好，持有者可以随时投票罢免
```

5.4.2 DPoS 的特点

- 优点：
  - 效率极高：每秒可处理数千甚至上万笔交易
  - 能耗极低：不需要大规模算力竞争
- 缺点：
  - 中心化程度高：只有 21 个节点控制网络
  - 投票率低：大多数代币持有者不参与投票
  - 去中心化程度降低——节点数量少，更容易被操控

5.5 PBFT —实用拜占庭容错

5.5.1 核心思想

通过多轮投票达成共识，只要恶意节点不超过总数的 1/3，系统就能正常工作。

5.5.2 三阶段协议

1. **Request**：客户端向主节点发送请求
2. **Pre-Prepare**：主节点验证请求，广播给所有节点
3. **Prepare**：节点验证并广播 Prepare 消息
4. **Commit**：节点收到足够多 Prepare 后，广播 Commit 消息
5. **Reply**：节点收到足够多 Commit 后，执行并回复客户端

PBFT 的容错能力：

- 总节点数 n，恶意节点数 f
- 系统能正常工作的条件：$n \geq 3f + 1$
- 即：恶意节点不超过总数的 1/3
- 4 个节点可以容忍 1 个恶意节点
- 10 个节点可以容忍 3 个恶意节点

5.5.3 PBFT 的特点

- 优点：
  - 即时确认：交易一旦写入就最终确定，不需要等待多个区块确认
  - 高吞吐量：每秒可处理数千笔交易
  - 确定性：不会出现分叉
- 缺点：
  - 扩展性差：节点数量增加，通信复杂度 $O(n^2)$ 急剧上升
  - 中心化趋势：通常用于联盟链，节点数量有限
  - 不适合公链：公链节点太多，PBFT 通信开销太大

重点：

- PBFT 适合联盟链（节点少、信任度较高）
- PoW/PoS 适合公链（节点多、无需许可）
- PBFT 是”投票”机制，PoW 是”算力竞争”机制
- 联盟链常用 PBFT（如 Hyperledger Fabric），公链用 PoW/PoS

5.6 Raft —非拜占庭容错共识

5.6.1 核心思想

Raft 是非拜占庭容错算法，假设所有节点都是诚实的，只处理节点故障（宕机、网络分区），不处理恶意节点。

5.6.2 三种角色

- **Leader**（领导者）：负责处理所有客户端请求，复制日志到 Follower
- **Follower**（跟随者）：接收并存储 Leader 的日志，被动响应
- **Candidate**（候选人）：Leader 宕机时，Follower 变成 Candidate，竞选新 Leader

5.6.3 Raft vs PBFT

特性	**Raft**	**PBFT**
容错类型	崩溃容错（非恶意）	拜占庭容错（含恶意节点）
节点假设	所有节点诚实	最多 1/3 节点恶意
通信复杂度	$O(n)$	$O(n^2)$
性能	更高（消息少）	较低（多轮投票）
适用场景	私有链、数据库复制	联盟链、跨机构协作
代表项目	etcd、Consul	Hyperledger Fabric

重点：Raft vs PBFT 的区别：

- Raft 假设节点只是故障，不会作恶（崩溃容错）
- PBFT 假设可能有节点作恶（拜占庭容错）
- 私有链/内网用 Raft，联盟链用 PBFT

5.7 共识算法深度解析

5.7.1 PoW 难度调整算法

比特币每 2016 个区块（约 2 周）调整一次难度，确保出块时间稳定在 10 分钟左右。

难度调整公式：

$$\text{新难度} = \text{旧难度} \times \frac{\text{实际时间}}{\text{目标时间}}$$

其中：

- 目标时间 = 2016 块 × 10 分钟/块 = 20160 分钟 = 2 周
- 实际时间 = 当前区块时间 - 2016 个区块前的时间
- 难度限制：调整幅度不超过 4 倍（防止剧烈波动）

Bits 字段编码：
区块头中的 Bits 字段是难度的紧凑表示（4 字节）。

```
Bits格式：
- 第1字节：指数（e）
- 后3字节：系数（c）

目标值 = c × 256^(e-3)

示例：
Bits = 0x1d00ffff
指数 e = 0x1d = 29
系数 c = 0x00ffff = 65535
目标值 = 65535 × 256^(29-3) = 65535 × 256^26

难度 = 最大目标值 / 当前目标值
最大目标值 = 0
    x00000000FFFF0000000000000000000000000000000000000000000000000000
```

难度调整的实际计算：

```
Python实现：

def calculate_new_difficulty(old_bits, actual_time, target_time
    =1209600):
    """
    计算新的难度目标
    old_bits: 当前bits值
    actual_time: 实际花费时间（秒）
    target_time: 目标时间（2周 = 1209600秒）
    """
    # 限制调整幅度为4倍
    if actual_time < target_time // 4:
        actual_time = target_time // 4
    if actual_time > target_time * 4:
        actual_time = target_time * 4

```

```
    # 计算新目标值
old_target = bits_to_target(old_bits)    new_target = (old_target * actual_time) // target_time

    # 确保不超过最大目标值
    max_target = 0x00000000FFFF0000000000000000000000000000000000000000000000000000

    if new_target > max_target:
        new_target = max_target

    return target_to_bits(new_target)

def bits_to_target(bits):
    """将bits转换为目标值"""
    exponent = (bits >> 24) & 0xff
    coefficient = bits & 0x007fffff
    return coefficient << (8 * (exponent - 3))

def target_to_bits(target):
    """将目标值转换为bits"""
    # 转换为紧凑格式
    if target == 0:
        return 0

    # 计算指数
    exponent = (target.bit_length() + 7) // 8

    # 计算系数（取最高3字节）
    if exponent <= 3:
        coefficient = target << (8 * (3 - exponent))
    else:
        coefficient = target >> (8 * (exponent - 3))

    # 确保系数不超过0x007fffff
    if coefficient & 0x00800000:
        coefficient >>= 8
        exponent += 1

    return (exponent << 24) | coefficient
```

5.7.2 PBFT 的数学证明

PBFT 能容忍最多 f 个恶意节点，需要满足 $n \geq 3f + 1$。

为什么需要 $n \geq 3f + 1$？
证明思路：
考虑最坏情况：系统需要确保诚实节点能达成一致，即使有 f 个恶意节点捣乱。
关键观察：

1. 在 Prepare 阶段，节点需要收到足够多的 Prepare 消息才能进入 Commit 阶段
2. 诚实节点必须确保：即使 f 个节点是恶意的，剩下的消息仍然来自诚实节点
3. 为了保证一致性，两个诚实节点收到的 Prepare 消息集合必须有交集

数学推导：
设总节点数为 n，恶意节点数为 f。

1. 诚实节点数 $= n - f$
2. 为了确保诚实节点能推进，需要收到至少 $2f + 1$ 个 Prepare 消息（包括自己）
3. 这意味着：$n - f \geq 2f + 1$（即使所有恶意节点都不发送消息，诚实节点仍够数）
4. 解得：$n \geq 3f + 1$

直观理解：

- 4 个节点，最多容忍 1 个恶意节点（诚实节点占 75%）
- 7 个节点，最多容忍 2 个恶意节点（诚实节点占 71%）
- 10 个节点，最多容忍 3 个恶意节点（诚实节点占 70%）

5.7.3 PoS 的罚没机制（Slashing）

以太坊 PoS 引入了罚没机制来防止恶意行为。

```
罚没条件：
1. 双重签名：在同一高度对两个不同区块签名
2. 环绕投票：投票违反了投票的"环绕"规则

罚没金额：
- 最低罚没：1 ETH
- 最高罚没：全部质押（32 ETH）
- 实际罚没取决于同时被惩罚的验证者数量
  （如果很多人同时作恶，罚没更多，防止协同攻击）
```

奖励和惩罚计算：

```
基础奖励公式：
base_reward = effective_balance * base_reward_factor // (
    base_rewards_per_epoch * sqrt(total_active_balance))

其中：
```

```
- effective_balance: 有效质押余额（上限32 ETH）
- base_reward_factor: 64（常数）- base_rewards_per_epoch: 4（每epoch的基础奖励数）
- total_active_balance: 全网活跃质押总量

出块奖励：
proposer_reward = base_reward // PROPOSER_REWARD_QUOTIENT
其中 PROPOSER_REWARD_QUOTIENT = 8

验证者年收益率估算：
年收益率 ≈ 基础收益率 × (1 - 离线惩罚比例)

示例：
- 全网质押1000万ETH
- 你的质押32 ETH
- 基础收益率约4.5%
- 如果100%在线，年收益约1.44 ETH
```

5.7.4 共识算法对比总结

特性	PoW	PoS	PBFT	Raft
容错类型	拜占庭容错	拜占庭容错	拜占庭容错	崩溃容错
恶意节点容忍	51% 算力	1/3 质押	1/3 节点	不适用
能耗	极高	极低	低	低
吞吐量	3-7 TPS	数千 TPS	数千 TPS	数万 TPS
最终性	概率性	确定性	即时	即时
去中心化	高	中	低	低
代表项目	比特币	以太坊 2.0	Fabric	etcd

6 智能合约

6.1 什么是智能合约？

智能合约（Smart Contract）是运行在区块链上的程序代码，可以自动执行合约条款。

智能合约的核心特点：

1. 自动执行：满足条件自动触发，无需人工干预
2. 不可篡改：一旦部署，代码无法修改
3. 透明公开：代码对所有参与者可见
4. 去中心化：在区块链网络的所有节点上运行

6.1.1 生活化类比

Nick Szabo 举了一个经典例子：自动售货机。

```
传统合同：
  1. 你和小明签合同："你给我100元，我给你一个商品"
  2. 你给了100元
  3. 小明可能反悔不给你商品
  4. 你需要去法院起诉

自动售货机（智能合约）：
  1. 你投入100元（满足条件）
  2. 你按下可乐按钮（发出指令）
  3. 机器自动吐出可乐（合同自动执行）
  ——整个过程不需要人工介入，不需要法院介入！
```

Nick Szabo 认为，法律合同的执行成本很高，而自动化的数字合同可以大幅降低成本。但 1994 年时没有可信的执行环境，直到以太坊出现。

6.2 以太坊和 EVM

6.2.1 以太坊的诞生

Vitalik Buterin（V 神）2013 年提出以太坊概念，2015 年正式上线。

以太坊 = 比特币的区块链 + 图灵完备的虚拟机

- 比特币的脚本系统：简单、非图灵完备（不能循环）
- 以太坊的 EVM：图灵完备，可以执行任意复杂逻辑

6.2.2 EVM（以太坊虚拟机）

EVM 是以太坊智能合约的运行环境：

- 每个以太坊节点都运行 EVM
- 智能合约代码在 EVM 中执行
- EVM 是沙箱环境，合约不能访问外部网络、文件系统等
- 合约之间可以互相调用

6.2.3 Gas 机制

执行智能合约需要消耗 **Gas**，防止无限循环和资源滥用。

```
简单转账：21000 Gas（固定）
执行一次算术运算：3 Gas
存储一个32字节数据：20000 Gas
执行一个合约函数：取决于函数复杂度

Gas计算示例：
  用户A调用一个复杂的DeFi合约：
```

```
  - 基本操作Gas: 50000
  - 存储写入Gas: 80000
  - Gas Price: 50 Gwei（0.00000005 ETH）
- 总费用 = 130000 × 50 Gwei = 0.0065 ETH
```

重点：Gas 是防止无限循环和资源滥用的机制。Gas Limit 是上限，防止意外超额。EVM（以太坊虚拟机）是智能合约的运行环境，与比特币的脚本系统不同。

6.3 Solidity 合约示例

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    uint256 storedData;

    function set(uint256 x) public {
        storedData = x;
    }

    function get() public view returns (uint256) {
        return storedData;
    }
}
```

调用这个合约：

```
set(42)  ->  交易被广播到网络
           ->  矿工打包到区块
           ->  所有节点执行set函数
           ->  storedData变成42
           ->  状态更新到区块链

get()    ->  本地查询，不消耗Gas
           ->  返回42
```

7 区块链的类型

7.1 公链（Public Blockchain）

- 特点：完全开放，任何人可以参与，无需许可
- 代表：比特币、以太坊
- 共识：PoW、PoS
- 优势：去中心化程度高，抗审查，全球可用

- 劣势：性能低，能耗高，监管困难

7.2 联盟链（Consortium Blockchain）

- 特点：多个机构共同维护，需要许可才能加入
- 代表：Hyperledger Fabric、R3 Corda、FISCO BCOS
- 共识：PBFT、Raft
- 优势：性能高，可监管，适合企业应用
- 劣势：去中心化程度较低

7.3 私链（Private Blockchain）

- 特点：单一机构控制，完全中心化
- 用途：企业内部审计、数据共享
- 优势：效率最高，完全可控
- 劣势：去中心化程度最低

重点：联盟链是中国主推方向，在政务、金融、供应链等领域广泛落地。应重点掌握联盟链的 PBFT/Raft 共识机制和实际应用场景。

8 区块链应用领域

8.1 数字货币与金融应用

8.1.1 比特币（Bitcoin）

比特币是第一个成功的加密货币，开创了区块链时代。

比特币的核心设计：

1. 总量恒定：2100 万枚，通过算法控制发行速度
2. 减半机制：每 4 年产量减半，控制通胀
3. 挖矿奖励：当前区块奖励为 6.25 BTC（2024 年减半后为 3.125 BTC）
4. 价值存储：被称为”数字黄金”，总量稀缺性支撑价值

内部资料

8.1.2 稳定币（Stablecoin）

稳定币是为了解决加密货币价格波动问题而设计的代币。

- 法币抵押型：USDT、USDC，1:1 锚定美元
  - 发行方持有等值美元储备
  - 用户可随时兑换回法币
  - 用于加密货币交易对定价
- 加密资产抵押型：DAI，通过超额抵押加密资产发行
  - 抵押率通常 >150%
  - 智能合约自动清算
  - 去中心化发行
- 算法型：通过算法调节供应量维持价格（风险较高）

8.1.3 央行数字货币（CBDC）

- 数字人民币（**e-CNY**）：中国央行发行的法定数字货币
  - 双层运营：央行-> 商业银行-> 公众
  - 可控匿名：小额匿名，大额可追溯
  - 双离线支付：无网络也能完成交易
  - 智能合约：支持条件支付、定向用途
- 与加密货币的区别：
  - CBDC 是法币，有国家信用背书
  - 中心化发行和管理
  - 不是投资品，是支付工具

重点：稳定币是连接传统金融和加密世界的桥梁。CBDC 是法币的数字化形式，与比特币等加密货币有本质区别。

8.2 资产数字化与供应链金融

8.2.1 资产数字化（Asset Tokenization）

将实物资产或权益转化为区块链上的数字代币。

内部资料

资产数字化的优势：

1. 流动性提升：传统非流动性资产（房地产、艺术品）可以碎片化交易
2. 降低门槛：投资者可以购买资产的一部分（如 1/1000 的房产份额）
3. 透明可追溯：资产所有权变更全程记录在链上
4. 降低中介成本：减少律师、公证等中介环节

可数字化的资产类型：

- 实物资产：房地产、黄金、艺术品、收藏品
- 金融资产：股票、债券、基金份额、保险
- 知识产权：专利、版权、商标
- 其他权益：会员权益、积分、门票

8.2.2 供应链金融

供应链金融解决中小企业融资难问题。

```
传统供应链金融的问题：
1. 核心企业确权难：核心企业不愿为上下游中小企业背书
2. 信息不对称：银行难以验证交易真实性
3. 融资成本高：中小企业信用不足，利率高
4. 资金周转慢：应收账款账期长

区块链解决方案：
1. 应收账款上链：核心企业应付账款转化为链上凭证
2. 可拆分流转：供应商可以将应收账款拆分转让给多级供应商
3. 智能合约自动执行：到期自动付款
4. 降低融资成本：银行可以直接验证交易真实性
```

典型案例：

- 蚂蚁链：”双链通”平台，服务超过 1 万家中小企业
- 微众银行：供应链金融平台，基于区块链的应收账款融资
- 京东智臻链：供应链溯源 + 金融一体化平台

重点：资产数字化通过 Token 化提升流动性。供应链金融通过区块链解决信任问题，让核心企业的信用可以传递给多级供应商。

8.3 政务与存证

8.3.1 电子政务

区块链在政务服务中的应用：

- 电子证照
  - 身份证、户口本、结婚证等上链存储
  - 跨部门共享，减少重复提交
  - 防篡改，提高可信度
  - 代表：广东省”粤省事”、浙江省”浙里办”
- 政务数据共享
  - 打破部门数据孤岛
  - 数据使用可追溯，保护隐私
  - 提高政务服务效率
- 公共资源交易
  - 招投标信息上链，防止暗箱操作
  - 确保公平公正

8.3.2 司法存证

区块链存证的优势：

1. 时间戳证明：精确记录证据产生时间
2. 防篡改：证据一旦上链无法修改
3. 司法认可：最高人民法院已明确区块链存证的法律效力
4. 降低成本：无需公证处，降低存证成本

典型案例：

- 北京互联网法院”天平链”：全国首个司法区块链平台
  - 已存证超过 1 亿条数据
  - 支持版权保护、网络侵权等案件
- 杭州互联网法院：司法区块链平台
- 广州互联网法院：”网通法链”平台

重点：区块链存证已被司法认可。最高法规定：当事人提交的区块链存证证据，经核验一致的，法院应当确认其真实性。

8.4 数字身份

- DID（去中心化身份），用户自主控制
- 可验证凭证（VC），保护隐私
- 应用：学历认证、职业资格认证

9 中国区块链发展与国密算法

9.1 国密算法概述

中国自主研发的一系列商用密码算法，包括：

- **SM2**：非对称加密算法（对应 ECDSA/RSA）
- **SM3**：Hash 算法（对应 SHA-256）
- **SM4**：对称加密算法（对应 AES）

9.2 SM2 —非对称加密算法

9.2.1 什么是 SM2?

SM2 是中国自主研发的非对称加密算法，功能类似于国际上的 RSA 和 ECDSA，但采用了不同的数学原理。

SM2 vs 国际算法的对应关系：

- SM2 ECDSA（椭圆曲线数字签名算法）——用于数字签名
- SM2 RSA ——用于加密
- 但 SM2 基于椭圆曲线，比 RSA 更高效、更安全

9.2.2 SM2 的核心概念

1. 密钥对

- 私钥：256 位随机数（64 个十六进制字符），必须保密
- 公钥：从私钥通过椭圆曲线计算得出，可以公开
  - 格式：04 + x 坐标 (32 字节) + y 坐标 (32 字节)
  - 总长度：65 字节（130 个十六进制字符）

2. 椭圆曲线参数 SM2 使用特定的椭圆曲线（sm2p256v1），定义了：

- 曲线方程：$y^2 = x^3 + ax + b$（特定的 a、b 值）
- 基点 G：曲线上的一个固定点
- 阶 n：一个大素数，确保安全性

9.2.3 SM2 的两大功能

功能 **1**：数字签名

用于证明消息确实来自私钥持有者，不可抵赖。

```
签名过程（简化理解）：
1. 计算消息Hash: e = Hash(message)   （使用SM3算法计算消息摘要）
2. 生成随机数k（每次签名都不同，必须保密）
3. 计算椭圆曲线点: (x1, y1) = k * G   （G是基点）
4. 计算r = (e + x1) mod n
5. 计算s = ((1 + d)^-1 * (k - r*d)) mod n
   其中d是私钥，n是曲线阶
6. 签名结果: (r, s) ——两个大整数

验证过程：
1. 计算消息Hash: e' = Hash(message)
2. 用公钥P和签名(r, s)验证等式是否成立
3. 验证通过 → 签名有效（消息确实来自私钥持有者）
4. 验证失败 → 签名无效或消息被篡改
```

功能 **2**：加密解密

用于保护数据机密性，只有私钥持有者能解密。

```
加密过程：
1. 生成随机数k
2. 计算椭圆曲线点: C1 = k * G（临时公钥）
3. 计算共享密钥: S = k * 接收方公钥
4. 用共享密钥加密消息 → C2
5. 计算校验值: C3 = SM3(S + message)
6. 密文: (C1, C2, C3)

解密过程：
1. 用私钥计算: S = 私钥 * C1
2. 用共享密钥解密C2得到消息
3. 验证C3是否正确
4. 正确 → 解密成功
```

SM2 考试重点：

- SM2 是基于椭圆曲线的非对称加密算法
- 私钥 256 位，公钥 512 位（04+x+y 格式）
- 签名结果是两个数 (r, s)
- 安全性基于椭圆曲线离散对数难题（ECDLP）
- 相同安全强度下，SM2 比 RSA 快 10 倍以上

9.3 SM3 —Hash 算法

9.3.1 什么是 SM3?

SM3 是中国自主研发的密码学 **Hash** 函数，功能类似于 SHA-256，但算法完全不同。

SM3 的核心特性：

- 输入：任意长度（理论上无限制）
- 输出：固定 256 位（64 个十六进制字符）
- 单向性：无法从 Hash 反推原始数据
- 雪崩效应：输入改变 1bit，输出完全不同
- 抗碰撞：极难找到两个不同输入产生相同 Hash

9.3.2 SM3 vs SHA-256

特性	**SM3**	**SHA-256**
输出长度	256 位	256 位
输出格式	64 个十六进制字符	64 个十六进制字符
内部结构	16 轮压缩函数	64 轮压缩函数
初始值	特定常数	自然常数的小数部分
安全性	国密标准	国际通用
相同输入结果	不同	不同

9.3.3 SM3 的计算过程（简化）

```
SM3计算步骤：

1. 消息填充：
   - 在消息后添加1个1和若干个0
   - 最后添加64位的消息长度
   - 确保总长度是512的倍数

2. 消息扩展：
   - 将消息分成512位的块
   - 每个块扩展成132个字（W0-W67, W'0-W'63）

3. 迭代压缩（16轮）：
   - 初始化8个寄存器（A-H）
   - 每轮更新寄存器的值
   - 使用布尔函数、移位运算、模加

4. 输出结果：
   - 将最终的8个寄存器值拼接
   - 得到256位的Hash值
```

实际应用示例：

```
Python使用SM3：
from gmssl import sm3

message = b"Hello SM3"
hash_value = sm3.sm3_hash(message)
print(f"SM3 Hash: {hash_value}")
# 输出：66c7f0f462eeedd9d1f2d46b5e9d0b3e8e8e8e8e8e8e8e8e8e8e8e8e8e8e8e8

对比SHA-256：
import hashlib
sha256_value = hashlib.sha256(message).hexdigest()
print(f"SHA-256: {sha256_value}")
# 输出：185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

结论：相同输入，SM3和SHA-256输出完全不同！
```

SM3 考试重点：

- SM3 输出 256 位（64 个十六进制字符）
- SM3 和 SHA-256 输出长度相同，但结果不同
- SM3 用于：数据完整性验证、数字签名中的 Hash 计算、区块链中的交易 Hash
- SM3 是国密标准，中国政府和企业优先使用

9.4 SM4 —对称加密算法

9.4.1 什么是 SM4?

SM4 是中国自主研发的对称加密算法，功能类似于 AES（高级加密标准），但算法设计完全不同。

对称加密的核心特点：

- 加密和解密使用同一个密钥
- 密钥必须安全地分发给通信双方
- 加密速度快，适合大数据量加密
- SM4 密钥长度：128 位（16 字节）
- SM4 分组长度：128 位（16 字节）

9.4.2 SM4 vs AES 对比

特性	**SM4**	**AES**
密钥长度	128 位（固定）	128/192/256 位
分组长度	128 位	128 位
加密轮数	32 轮	10/12/14 轮
结构	非平衡 Feistel 网络	替换-置换网络（SPN）
S 盒	固定 8 位 S 盒	可逆 8 位 S 盒

9.4.3 SM4 的加密过程

```
SM4加密步骤（简化）：

1. 密钥扩展：
   - 输入：128位密钥
   - 输出：32个轮密钥（每轮一个）
   - 使用FK常数和CK常数进行扩展

2. 轮函数（32轮）：
   每轮操作：
   - X = 当前状态（4个字）
   - 非线性变换τ：使用S盒进行字节替换
   - 线性变换L：循环移位和异或
   - 轮密钥加：与轮密钥异或
   - 更新状态

3. 反序变换：
   - 32轮结束后，将结果反序
   - 得到128位密文
```

9.4.4 SM4 的工作模式

```
ECB模式（电子密码本）——不推荐：
┌──────────┐  ┌──────────┐  ┌──────────┐
│ 明文块1 │  │ 明文块2 │  │ 明文块3 │
```

```
└──────┬──────┘   └──────┬──────┘   └──────┬──────┘
       │                 │                 │
       │                 │                 │
   ┌───┴───┐         ┌───┴───┐         ┌───┴───┐
   │ SM4   │         │ SM4   │         │ SM4   │   ← 相同密钥，独立加密
   │加密   │         │加密   │         │加密   │
   └───┬───┘         └───┬───┘         └───┬───┘
       │                 │                 │
   ┌───┴──────────┐  ┌───┴──────────┐  ┌───┴──────────┐
   │ 密文块1      │  │ 密文块2      │  │ 密文块3      │
   └──────────────┘  └──────────────┘  └──────────────┘

缺点：相同明文块产生相同密文块，不安全

CBC模式（密码分组链接）——推荐：
┌──────────────┐   ┌──────────────┐   ┌──────────────┐
│ 明文块1      │   │ 明文块2      │   │ 明文块3      │
└──────┬───────┘   └──────┬───────┘   └──────┬───────┘
       │                  │                  │
       │  IV(初始向量)    │                  │
       │     │            │     │            │
       └─────┼────────────┼─────┼────────────┘
             ▼            │     ▼            │
          ┌──┴──┐         │  ┌──┴──┐         │
          │XOR  │         │  │XOR  │         │
          └──┬──┘         │  └──┬──┘         │
             │            │     │            │
          ┌──┴──┐         │  ┌──┴──┐         │
          │SM4  │         │  │SM4  │         │
          │加密 │         │  │加密 │         │
          └──┬──┘         │  └──┬──┘         │
             │            └─────┼────────────┘
             │                  │
       ┌─────┴────────┐   ┌─────┴────────┐
       │ 密文块1      │   │ 密文块2      │
       └──────────────┘   └──────────────┘

优点：相同明文产生不同密文，更安全
需要：随机IV（初始向量），每次加密都不同
```

9.4.5 SM4 的实际应用

```
Python使用SM4（使用gmssl库）：

from gmssl.sm4 import CryptSM4, SM4_ENCRYPT, SM4_DECRYPT

# 密钥（16字节）
key = b'0123456789abcdef'

# 初始向量（16字节，CBC模式需要）
iv = b'abcdef0123456789'

```

```
# 创建SM4对象
sm4_crypt = CryptSM4()
# 加密sm4_crypt.set_key(key, SM4_ENCRYPT)
plaintext = b'Hello, SM4!'
# 填充到16字节倍数
padding_len = 16 - len(plaintext) % 16
plaintext += bytes([padding_len] * padding_len)
ciphertext = sm4_crypt.crypt_cbc(iv, plaintext)
print(f"密文: {ciphertext.hex()}")

# 解密
sm4_crypt.set_key(key, SM4_DECRYPT)
decrypted = sm4_crypt.crypt_cbc(iv, ciphertext)
# 去除填充
padding_len = decrypted[-1]
decrypted = decrypted[:-padding_len]
print(f"解密: {decrypted.decode()}")
```

SM4 考试重点：

- SM4 是对称加密算法，加密解密用同一个密钥
- 密钥长度 128 位，分组长度 128 位，32 轮加密
- 推荐使用 CBC 模式（需要 IV），不推荐 ECB 模式
- SM4 适合加密大量数据（比非对称加密快 100 倍以上）
- 应用场景：数据传输加密、文件加密、数据库加密

9.5 国密算法总结

算法	类型	对应国际算法	主要用途
SM2	非对称加密	ECDSA/RSA	数字签名、密钥交换
SM3	Hash 函数	SHA-256	数据完整性、签名 Hash
SM4	对称加密	AES	大数据量加密

国密算法的优势：

1. 自主可控：不依赖国外技术，保障国家安全
2. 性能优化：针对中国硬件环境优化
3. 合规要求：中国政府和企业项目必须使用
4. 安全性：经过严格密码学分析和测试
5. 生态完善：支持硬件加速（加密卡、加密芯片）

10 数据结构基础

10.1 链表（Linked List）

```
普通链表：
  节点A -> 节点B -> 节点C -> 节点D
  指针存储下一个节点的内存地址

Hash指针链表（区块链）：
  节点A -> 节点B -> 节点C -> 节点D
  节点A的"指针" = Hash(节点B的内容)

  优势：即使有人修改了节点C的数据，
  节点D中存储的Hash(节点C的内容) 立刻失效，
  整个篡改行为被发现。
```

重点：区块链使用链表结构，每个区块通过 Hash 指针指向前一个区块。Hash 指针 = 数据内容 Hash + 位置指针，即使数据移动，Hash 值仍然有效，保证链的完整性。

10.2 树（Tree）

10.2.1 二叉树

每个节点最多有两个子节点。

10.2.2 Merkle Tree

见第 3 章，区块链核心数据结构。

10.2.3 MPT（Merkle Patricia Tree）

以太坊使用的数据结构，结合了：

- Merkle Tree：验证数据完整性
- Patricia Trie：高效存储和检索

重点：以太坊使用 MPT 而非比特币的简单 Merkle Tree，因为 MPT 支持增删改查操作（而 Merkle Tree 主要用于验证完整性）。MPT 是理解以太坊区块结构的关键。

10.3 Merkle Tree 的数学性质

10.3.1 Merkle Tree 的高度与证明大小

Merkle Tree 的效率体现在验证交易存在性的复杂度上。

数学分析：
设交易数量为 n，则：

- Merkle Tree 高度 $= \lceil \log_2 n \rceil$
- 验证一笔交易需要的 Hash 数量 = 高度 $= \lceil \log_2 n \rceil$

对比：

交易数量	**Tree** 高度	验证所需 **Hash** 数
1024 (2^{10})	10	10
65536 (2^{16})	16	16
100 万 ($\approx 2^{20}$)	20	20
10 亿 ($\approx 2^{30}$)	30	30

结论：即使区块包含数百万笔交易，验证一笔交易也只需要约 20 个 Hash 值（约 640 字节），这是 SPV 轻节点可行的数学基础。

10.3.2 Merkle Proof 的构造与验证

```
Merkle Proof构造算法：

输入：交易tx，其在叶子节点中的索引i，所有叶子节点leaves[]
输出：Merkle Proof（路径上的兄弟节点Hash列表）

function getMerkleProof(tx, i, leaves):
    proof = []
    current_hash = hash(tx)

    # 逐层向上计算
    while len(leaves) > 1:
        # 如果是奇数，复制最后一个
        if len(leaves) % 2 == 1:
```

```
            leaves.append(leaves[-1])
# 找到当前节点的兄弟节点        if i % 2 == 0:  # 当前是左节点
            sibling = leaves[i + 1]
            proof.append(('right', sibling))  # 兄弟在右边
            current_hash = hash(current_hash + sibling)
        else:  # 当前是右节点
            sibling = leaves[i - 1]
            proof.append(('left', sibling))  # 兄弟在左边
            current_hash = hash(sibling + current_hash)

        # 计算下一层的节点
        next_level = []
        for j in range(0, len(leaves), 2):
            next_level.append(hash(leaves[j] + leaves[j+1]))
        leaves = next_level
        i = i // 2

    return proof

Merkle Proof验证算法:

输入: 交易tx, Merkle Proof, Merkle Root
输出: 验证结果 (True/False)

function verifyMerkleProof(tx, proof, root):
    current_hash = hash(tx)

    for direction, sibling_hash in proof:
        if direction == 'left':  # 兄弟在左边
            current_hash = hash(sibling_hash + current_hash)
        else:  # 兄弟在右边
            current_hash = hash(current_hash + sibling_hash)

    return current_hash == root
```

10.3.3 UTXO 模型的数学分析

UTXO 模型与账户模型的对比可以从数学角度分析。

UTXO 集合的状态空间：
设：

- U = UTXO 集合
- t = 时间（区块高度）
- $u_i = (txid, vout, value, script)$ 表示单个 UTXO

系统状态可以表示为：

$$S_t = \{u_1, u_2, ..., u_n\}$$

状态转移函数：

$$S_{t+1} = T(S_t, tx_{t+1})$$

其中 T 是状态转移函数：

- 输入：消耗 UTXO（从集合中移除）
- 输出：创建新 UTXO（添加到集合）

账户模型的状态空间：

$$S_t = \{(addr_1, balance_1), (addr_2, balance_2), ..., (addr_m, balance_m)\}$$

状态转移更简单：

$$balance_{sender} -= amount$$

$$balance_{receiver} += amount$$

UTXO vs 账户模型的数学对比：

特性	**UTXO 模型**	账户模型
状态表示	未花费输出集合	地址-余额映射
交易验证	需验证引用的 UTXO 存在且未花费	只需验证发送方余额充足
并行性	天然支持（无关联的 UTXO 可并行花费）	需要额外机制（nonce 序列化）
隐私性	较好（每次交易使用新地址）	较差（地址重复使用）
复杂度	O(|UTXO 集合 |)	O(| 账户数 |)

10.3.4 比特币地址生成的完整流程

```
比特币地址生成（P2PKH）的数学流程：

1. 生成私钥（32字节随机数）：
   private_key = random(256 bits)

2. 使用secp256k1曲线生成公钥：
```

```
   public_key = private_key * G
   其中G是曲线基点
公钥格式（未压缩，65字节）：    04 || x坐标(32字节) || y坐标(32字节)

   公钥格式（压缩，33字节）：
   02/03 || x坐标(32字节)   （02表示y为偶数，03表示y为奇数）

3. 计算公钥Hash（SHA-256然后RIPEMD-160）：
   hash160 = RIPEMD-160(SHA-256(public_key))
   结果：20字节

4. 添加版本前缀（主网0x00）：
   versioned = 0x00 || hash160

5. 计算校验和（前4字节）：
   checksum = SHA-256(SHA-256(versioned))[0:4]

6. 组合并Base58编码：
   binary = versioned || checksum
   address = Base58Encode(binary)

完整示例：
私钥（十六进制）：
E9873D79C6D87DC0FB6A5778633389F4453213303DA61F20BD67FC233AA33262

公钥（未压缩）：
0450863
   AD64A87AE8A2FE83C1AF1A8403CB53F53E486D8511DAD8A04887E5B23522CD470243453A29


Hash160：
010966776006953D5567439E5E39F86A0D273BEE

地址：
16UwLL9Risc3QfPqBUvKofHmBQ7wMtjvM
```

10.3.5 以太坊地址生成

```
以太坊地址生成流程：

1. 生成私钥（同比特币）：
   private_key = random(256 bits)

2. 使用secp256k1生成公钥：
   public_key = private_key * G

   以太坊只使用未压缩公钥（64字节，去掉04前缀）：
   public_key = x(32字节) || y(32字节)
```

```

3. 计算Keccak-256 Hash:
hash = Keccak-256(public_key)    结果: 32字节

4. 取后20字节作为地址:
   address = hash[12:32]  # 取最后20字节

5. 添加0x前缀:
   ethereum_address = "0x" + hex(address)

注意: 以太坊地址没有校验和(早期版本),
后来EIP-55引入了大小写校验和编码

示例:
私钥:
df57089febbacf7ba0bc227dafbffa9fc08a93fdc68e1e42411a14efcf23656e

公钥:
044bc43e6a7c61b74e2a9d6194eb5c3e5f7e5e5e5e5e5e5e5e5e5e5e5e5e5e5e
5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e5e

地址:
0x8ba1f109551bD432803012645Hac136c82C3e5c5
```

11 链上身份（DID）

11.1 什么是 DID?

DID（Decentralized Identifier，去中心化标识符）是用户自主控制的数字身份，不依赖中心化机构（如政府、公司）。

11.1.1 DID 的格式

```
did:example:123456789abcdefghi
├── did: 方案前缀
├── example: DID方法名(指定操作DID的区块链/网络)
└── 123456789abcdefghi: 唯一标识符(由DID方法生成)
```

11.1.2 DID 的生成

用户通过加密算法生成 DID 和密钥对，DID 本身不包含任何个人身份信息（隐私设计）。

```
DID密钥生成流程:
1. 生成ECDSA secp256k1密钥对(或SM2)
   私钥: 0x7c9e6a5d...
   公钥: 0x04a8e7b6...
2. 计算公钥Hash: SHA256(公钥) -> RIPEMD160 -> Base58
3. 加上方法前缀: did:ethr:0x1234...
```

```
4. DID文档存储在以太坊或IPFS上
```

11.1.3 DID 文档（DID Document）

DID 文档描述了与 DID 关联的公钥、服务端点等信息，存储在区块链或去中心化存储上。

```
DID Document示例:
{
  "@context": "https://www.w3.org/ns/did/v1",
  "id": "did:ethr:0x1234abcd...",
  "verificationMethod": [{
    "id": "did:ethr:0x1234#keys-1",
    "type": "EcdsaSecp256k1",
    "controller": "did:ethr:0x1234...",
    "publicKeyHex": "02a8e7b6..."
  }],
  "authentication": ["did:ethr:0x1234...#keys-1"],
  "service": [{
    "type": "LinkedDomains",
    "serviceEndpoint": "https://..."
  }]
}
```

DID 文档可以更新的操作（更改公钥、服务端点），但 DID 标识符本身保持不变。

11.2 可验证凭证（VC）

VC（Verifiable Credential，可验证凭证）是由权威机构签发的、关于某个主体的数字化声明。

```
VC示例: 学历证书
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "type": ["VerifiableCredential", "UniversityDegreeCredential"],
  "issuer": "did:ethr:0xuniversity...",
  "issuanceDate": "2024-06-01",
  "credentialSubject": {
    "id": "did:ethr:0xstudent...",
    "degree": {
      "type": "BachelorDegree",
      "name": "计算机科学",
      "university": "某某大学"
    }
  },
  "proof": {
    "type": "EcdsaSecp256k1Signature2019",
    "created": "2024-06-01",
    "proofValue": "..."
  }
}
```

VC 的核心特点：

1. 可验证：任何人都可以验证签发者身份和签名
2. 防篡改：任何修改都会导致签名失效
3. 隐私保护：可以选择性披露（只证明”我有本科学历”，不透露具体学校）
4. 自主控制：用户自己存储和管理凭证，不需要中心数据库

重点：DID = 去中心化标识符，由用户自己控制，不依赖中心化机构。三个核心组件：DID 标识符、DID 文档（存公钥）、可验证凭证（VC）。DID 文档存储在区块链或 IPFS 上。

12 实操技能备考要点

12.1 实操技能要点

本章节主要涵盖实操技能相关知识点：

1. 数据结构的基本原理及其在区块链中的应用
2. 主流密码学算法在区块链身份标识方面的应用

12.2 Python 编程要点

12.2.1 Hash 计算

```python
import hashlib

# SHA-256计算
def sha256_hash(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

# 使用示例
result = sha256_hash("Hello Blockchain")
print(f"SHA-256: {result}")
# 输出: SHA-256: 7
    f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d9069
```

12.2.2 Merkle Root 计算

```
import hashlib
def sha256(data):    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

def merkle_root(txs):
    """计算Merkle Root"""
    if not txs:
        return sha256("")

    # 计算所有交易的Hash
    hashes = [sha256(tx) for tx in txs]

    # 逐层计算，直到只剩一个Hash
    while len(hashes) > 1:
        # 如果奇数个，复制最后一个
        if len(hashes) % 2 == 1:
            hashes.append(hashes[-1])

        # 两两配对计算父Hash
        parent_hashes = []
        for i in range(0, len(hashes), 2):
            parent = sha256(hashes[i] + hashes[i+1])
            parent_hashes.append(parent)

        hashes = parent_hashes

    return hashes[0]

# 测试
txs = ["tx1", "tx2", "tx3", "tx4"]
print(f"Merkle Root: {merkle_root(txs)}")
```

12.2.3 简单区块链实现

```
import hashlib
import time

class Block:
    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.mine_block()

```

```
    def calculate_hash(self):
        value = str(self.index) + str(self.timestamp) + \
str(self.data) + str(self.previous_hash) + \                str(
                self.nonce)
        return hashlib.sha256(value.encode()).hexdigest()

    def mine_block(self, difficulty=4):
        """PoW挖矿"""
        target = "0" * difficulty
        while True:
            self.hash = self.calculate_hash()
            if self.hash.startswith(target):
                return self.hash
            self.nonce += 1

class Blockchain:
    def __init__(self):
        self.chain = [self.create_genesis_block()]

    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    def get_latest_block(self):
        return self.chain[-1]

    def add_block(self, data):
        previous_hash = self.get_latest_block().hash
        new_block = Block(len(self.chain), data, previous_hash)
        self.chain.append(new_block)

    def is_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i-1]

            if current.hash != current.calculate_hash():
                return False
            if current.previous_hash != previous.hash:
                return False
        return True

# 使用示例
blockchain = Blockchain()
blockchain.add_block("First Transaction")
blockchain.add_block("Second Transaction")

print(f"区块链有效: {blockchain.is_valid()}")
for block in blockchain.chain:
    print(f"区块{block.index}: {block.hash[:20]}...")
```

12.3 区块链浏览器实操

12.3.1 比特币浏览器使用

以 blockchain.com 为例，学习如何查询交易信息。

```
实操步骤：
1. 打开 blockchain.com/explorer
2. 在搜索框输入交易ID（TxID）
   示例：
      4a5e1e4baab89f3a32518a88c31bc87f618f76673e2cc77ab2127b7afdeda33b

   （这是比特币创世区块的第一笔交易）

3. 查看交易详情：
   - 交易哈希（Transaction Hash）
   - 区块高度（Block Height）
   - 确认数（Confirmations）
   - 发送方/接收方地址
   - 交易金额
   - 交易费用
   - 时间戳

4. 查看区块信息：
   - 区块哈希
   - 前一区块哈希（形成链式结构）
   - Merkle Root
   - Nonce
   - 难度目标
   - 包含的交易数量
```

12.3.2 以太坊浏览器使用

以 etherscan.io 为例。

```
实操步骤：
1. 打开 etherscan.io
2. 查询账户地址：
   示例：0x742d35Cc6634C0532925a3b844Bc9e7595f0bEb

3. 查看账户信息：
   - ETH余额
   - 交易数量
   - 持有的代币（Token）
   - 交易历史

4. 查询智能合约：
   - 合约代码（Verified Source Code）
   - 合约ABI
   - 合约创建交易
   - 合约调用记录
```

```

5. 查看Gas费用:
- Gas Price（Gwei）    - Gas Limit
   - 实际消耗的Gas
   - 交易总费用 = Gas Used × Gas Price
```

重点：区块链浏览器是查看链上数据的窗口。比特币浏览器主要看 UTXO 和区块信息，以太坊浏览器主要看账户、智能合约和 Gas 费用。

12.4 数字签名实操

12.4.1 ECDSA 签名与验证（Python 实现）

```python
import hashlib
import ecdsa
from ecdsa import SigningKey, VerifyingKey, SECP256k1

def generate_keypair():
    """生成密钥对"""
    # 生成私钥
    private_key = SigningKey.generate(curve=SECP256k1)
    # 从私钥生成公钥
    public_key = private_key.get_verifying_key()
    return private_key, public_key

def sign_message(private_key, message):
    """使用私钥签名消息"""
    # 计算消息Hash
    message_hash = hashlib.sha256(message.encode()).digest()
    # 签名
    signature = private_key.sign(message_hash, hashfunc=hashlib.sha256)
    return signature

def verify_signature(public_key, message, signature):
    """使用公钥验证签名"""
    try:
        message_hash = hashlib.sha256(message.encode()).digest()
        # 验证签名
        public_key.verify(signature, message_hash, hashfunc=hashlib.sha256)
        return True
    except ecdsa.BadSignatureError:
        return False

# 实操示例
if __name__ == "__main__":
```

```
    # 1. 生成密钥对
private_key, public_key = generate_keypair()    print(f"私钥: {private_key.to_string().hex()}")
    print(f"公钥: {public_key.to_string().hex()}")

    # 2. 要签名的消息
    message = "转账给Alice 1个比特币"
    print(f"\n原始消息: {message}")

    # 3. 签名
    signature = sign_message(private_key, message)
    print(f"签名: {signature.hex()}")

    # 4. 验证签名
    is_valid = verify_signature(public_key, message, signature)
    print(f"\n签名验证结果: {is_valid}")

    # 5. 篡改消息后再验证
    tampered_message = "转账给Bob 100个比特币"
    is_valid_tampered = verify_signature(public_key, tampered_message, signature)
    print(f"篡改后验证结果: {is_valid_tampered}")
```

12.4.2 SM2 签名与验证（国密算法）

```
from gmssl import sm2, func

def sm2_sign_verify():
    """SM2签名与验证示例"""
    # 密钥对（十六进制表示）
    private_key = '59e2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42601e2'
    public_key = '04' + '8d1d8c5df8e4286e0c8d25b602b5503e0ad4c8b6c374b8d9b8f2e8e5e5e5e5e' + \
                 '9f2e5de1614f0f830655f4b16b5903f247b701894c1f21f6f6e0631f42'

    # 创建SM2对象
    sm2_crypt = sm2.CryptSM2(public_key=public_key, private_key=private_key)

    # 要签名的消息
    message = b"转账给Alice 100元"

    # 签名
    signature = sm2_crypt.sign(message, private_key)
```

```
    print(f"SM2签名: {signature.hex()}")

# 验证签名    result = sm2_crypt.verify(signature, message)
    print(f"SM2签名验证结果: {result}")

    return result

# 注意：实际使用时需要安装gmssl库
# pip install gmssl
```

签名验证的核心逻辑：

1. 签名过程：私钥 + 消息 Hash → 签名值 (r, s)
2. 验证过程：公钥 + 消息 Hash + 签名值 → 验证通过/失败
3. 安全性：没有私钥无法生成有效签名，但任何人都可以用公钥验证
4. 不可抵赖：只有私钥持有者能生成签名，签名者无法否认

12.5 备考建议

1. 理解原理：不要死记硬背，理解每个算法的原理和应用场景
2. 动手实践：至少实现一遍 SHA-256、Merkle Tree、简单区块链
3. 熟悉 **Python**：掌握基本语法、hashlib 库的使用
4. 编码规范：代码缩进、命名、注释
5. 时间分配：理论 45 分钟，实操 60 分钟，合理分配时间

内部资

13 附录 A：核心术语速查表

术语	解释
区块链	分布式账本技术，数据不可篡改
Hash	单向函数，任意输入产生固定长度输出
SHA-256	安全 Hash 算法，输出 256 位
Merkle Tree	二叉树结构，用于快速验证数据完整性
UTXO	未花费交易输出，比特币的记账模型
PoW	工作量证明，通过算力竞争记账权
PoS	权益证明，通过持币量和时间选择验证者
PBFT	实用拜占庭容错，联盟链常用共识
智能合约	运行在区块链上的自动执行程序
DID	去中心化身份标识符
VC	可验证凭证
SM2/SM3/SM4	中国国密算法
Gas	以太坊执行操作的费用单位

内部资料

14 附录 B：知识体系思维导图

重点速查（按模块整理）

【必须记忆】

- Hash 特性：单向、雪崩、抗碰撞
- SHA-256 输出：64 个十六进制字符 = 256 位
- ECC vs RSA：256 位 ECC ≈ 3072 位 RSA
- 私钥签名、公钥验证
- 区块头 6 个字段：Version、PrevHash、Merkle Root、Timestamp、Bits、Nonce
- UTXO 模型：输入引用过去的输出，产生新的输出
- PoW：算力竞争，找到 Nonce 使 Hash < 目标值
- PoS：质押代币，随机选择验证者
- PBFT：三阶段投票，容忍 < 1/3 恶意节点
- 联盟链 vs 公链：PBFT/Raft vs PoW/PoS
- SM2/SM3/SM4：国密非对称加密、Hash、对称加密
- DID 三要素：标识符、文档、可验证凭证

【理解即可】

- 拜占庭将军问题：叛徒不超过 1/3 时忠诚将军可达成一致
- PoS 根据持币量和时间选择记账者，不消耗算力
- Merkle Tree：根 Hash 代表所有交易，任意一笔修改导致根变化
- 联盟链用 PBFT/Raft，公链用 PoW/PoS
- DID 三要素：DID 标识符 + DID 文档 + 可验证凭证（VC）

【实操相关】

- SHA-256 计算：`hashlib.sha256(data).hexdigest()`
- 区块链结构：区块头（Hash 指针链接）+ 区块体（交易列表）
- Merkle Root 计算：逐层两两 Hash，直到只剩一个根 Hash

15 附录 C：推荐学习资源

- 比特币白皮书：《Bitcoin: A Peer-to-Peer Electronic Cash System》（中本聪）
- 以太坊白皮书：《A Next-Generation Smart Contract and Decentralized Application Platform》（Vitalik Buterin）
- 国密标准：GM/T 0003-2012（SM2）、GM/T 0004-2012（SM3）、GM/T 0002-2012（SM4）
- 在线工具：`blockchain.com/explorer`（比特币浏览器）、`etherscan.io`（以太坊浏览器）
- **Python** 库：`hashlib`（标准库）、`gmssl`（国密算法）

16 附录 D：实操环境搭建指南

16.1 Python 环境准备

1. 安装 **Python 3.8+**：从官网下载安装 `python.org`
2. 安装必要库：

```
pip install hashlib gmssl pycryptodome
```

3. 验证安装：

```
import hashlib
print(hashlib.sha256(b"test").hexdigest())
```

16.2 区块链浏览器使用

1. 比特币浏览器：`blockchain.com/explorer`
   - 输入交易 ID 查询交易详情
   - 查看区块高度、时间戳、交易数量
2. 以太坊浏览器：`etherscan.io`
   - 查询账户余额和交易历史
   - 查看智能合约代码

17 附录 E：模拟试题

17.1 理论知识题

一、选择题（每题 **5** 分）

内部资料

1. SHA-256 算法的输出长度是 ()

   A 128 位
   B 256 位
   C 512 位
   D 1024 位

2. 比特币使用的共识算法是 ()

   A PoS
   B PBFT
   C PoW
   D Raft

3. PBFT 算法能容忍的恶意节点比例最多为 ()

   A 1/2
   B 1/3
   C 2/3
   D 1/4

4. 以下哪个是中国国密非对称加密算法 ()

   A RSA
   B ECDSA
   C SM2
   D AES

二、简答题（每题 **10** 分）

1. 简述区块链的不可篡改性是如何通过密码学实现的。
2. 解释 UTXO 模型与账户余额模型的区别。
3. 说明 Merkle Tree 在区块链中的作用。

17.2 实操技能题

三、编程题（每题 **20** 分）

1. 编写 Python 函数计算给定字符串的 SHA-256 哈希值，并验证雪崩效应。
   参考答案：

```python
import hashlib

def sha256_hash(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

# 验证雪崩效应
h1 = sha256_hash("blockchain")
h2 = sha256_hash("blockchaiN")
print(f"原始: {h1}")print(f"修改: {h2}")
print(f"不同位数: {sum(c1 != c2 for c1, c2 in zip(h1, h2))}")
```

2. 实现简单的 Merkle Root 计算函数。
   参考答案：

```python
import hashlib

def sha256(data):
    if isinstance(data, str):
        data = data.encode('utf-8')
    return hashlib.sha256(data).hexdigest()

def merkle_root(txs):
    hashes = [sha256(tx) for tx in txs]
    while len(hashes) > 1:
        if len(hashes) % 2 == 1:
            hashes.append(hashes[-1])
        parent_hashes = []
        for i in range(0, len(hashes), 2):
            parent = sha256(hashes[i] + hashes[i+1])
            parent_hashes.append(parent)
        hashes = parent_hashes
    return hashes[0]

# 测试
txs = ["tx1", "tx2", "tx3", "tx4"]
print(f"Merkle Root: {merkle_root(txs)}")
```

17.3 答案速查

题号	答案	题号	答案
1	B	3	B
2	C	4	C

18 算法实现详解

本章提供核心算法的完整 Python 实现，供学生课后深入学习和实践。

18.1 完整区块链实现

18.1.1 基础数据结构

```python
import hashlib
import json
import time
from typing import List, Dict, Optional

class Transaction:
    """交易类"""
    def __init__(self, sender: str, recipient: str, amount: float):
        self.sender = sender
        self.recipient = recipient
        self.amount = amount
        self.timestamp = time.time()

    def to_dict(self) -> Dict:
        return {
            'sender': self.sender,
            'recipient': self.recipient,
            'amount': self.amount,
            'timestamp': self.timestamp
        }

    def hash(self) -> str:
        """计算交易Hash"""
        tx_string = json.dumps(self.to_dict(), sort_keys=True)
        return hashlib.sha256(tx_string.encode()).hexdigest()


class Block:
    """区块类"""
    def __init__(self, index: int, transactions: List[Transaction],
                 previous_hash: str, difficulty: int = 4):
        self.index = index
        self.timestamp = time.time()
        self.transactions = transactions
        self.previous_hash = previous_hash
        self.difficulty = difficulty
        self.nonce = 0
        self.hash = self.mine_block()

    def to_dict(self) -> Dict:
        """转换为字典（用于Hash计算）"""
        return {
            'index': self.index,
            'timestamp': self.timestamp,
            'transactions': [tx.to_dict() for tx in self.
```

```
                transactions],
'previous_hash': self.previous_hash,            'difficulty':
            self.difficulty,
            'nonce': self.nonce
        }

    def calculate_hash(self) -> str:
        """计算区块Hash"""
        block_string = json.dumps(self.to_dict(), sort_keys=True)
        return hashlib.sha256(block_string.encode()).hexdigest()

    def mine_block(self) -> str:
        """PoW挖矿"""
        target = "0" * self.difficulty
        print(f"挖矿中... 难度: {self.difficulty}")

        start_time = time.time()
        while True:
            self.nonce += 1
            block_hash = self.calculate_hash()

            if block_hash.startswith(target):
                elapsed = time.time() - start_time
                print(f"找到有效Hash! Nonce: {self.nonce}")
                print(f"Hash: {block_hash}")
                print(f"耗时: {elapsed:.2f}秒")
                return block_hash

            # 每100万次尝试打印一次进度
            if self.nonce % 1000000 == 0:
                print(f"已尝试 {self.nonce} 次...")


class Blockchain:
    """区块链类"""
    def __init__(self, difficulty: int = 4):
        self.chain: List[Block] = []
        self.difficulty = difficulty
        self.pending_transactions: List[Transaction] = []
        self.mining_reward = 100.0  # 挖矿奖励

        # 创建创世区块
        self.create_genesis_block()

    def create_genesis_block(self):
        """创建创世区块"""
        genesis_block = Block(0, [], "0", self.difficulty)
        self.chain.append(genesis_block)
        print("创世区块已创建！")

```

```
    def get_latest_block(self) -> Block:
"""获取最新区块"""        return self.chain[-1]

    def add_transaction(self, transaction: Transaction):
        """添加待确认交易"""
        # 验证交易
        if not transaction.sender or not transaction.recipient:
            raise ValueError("交易必须包含发送方和接收方")
        if transaction.amount <= 0:
            raise ValueError("交易金额必须大于0")

        self.pending_transactions.append(transaction)
        print(f"交易已添加到待确认池: {transaction.hash()
            [:16]}...")

    def mine_pending_transactions(self, miner_address: str):
        """挖矿：将待确认交易打包成区块"""
        # 添加挖矿奖励交易
        reward_tx = Transaction("network", miner_address, self.
            mining_reward)
        self.pending_transactions.insert(0, reward_tx)

        # 创建新区块
        new_block = Block(
            index=len(self.chain),
            transactions=self.pending_transactions,
            previous_hash=self.get_latest_block().hash,
            difficulty=self.difficulty
        )

        self.chain.append(new_block)
        self.pending_transactions = []  # 清空待确认交易
        print(f"区块 {new_block.index} 已添加到链上！")

    def get_balance(self, address: str) -> float:
        """计算账户余额"""
        balance = 0.0

        for block in self.chain:
            for tx in block.transactions:
                if tx.recipient == address:
                    balance += tx.amount
                if tx.sender == address:
                    balance -= tx.amount

        return balance

    def is_chain_valid(self) -> bool:
        """验证区块链完整性"""
        for i in range(1, len(self.chain)):
```

```
            current_block = self.chain[i]
previous_block = self.chain[i-1]
            # 验证当前区块Hash
            if current_block.hash != current_block.calculate_hash
                ():
                print(f"区块 {i} 的Hash无效！")
                return False

            # 验证链式连接
            if current_block.previous_hash != previous_block.hash
                :
                print(f"区块 {i} 与前一个区块的连接断开！")
                return False

            # 验证PoW
            target = "0" * current_block.difficulty
            if not current_block.hash.startswith(target):
                print(f"区块 {i} 的工作量证明无效！")
                return False

        return True

    def print_chain(self):
        """打印整个区块链"""
        print("\n" + "="*60)
        print("区块链内容")
        print("="*60)

        for block in self.chain:
            print(f"\n区块 #{block.index}")
            print(f"  Hash: {block.hash}")
            print(f"  前一区块Hash: {block.previous_hash}")
            print(f"  时间戳: {time.strftime('%Y-%m-%d %H:%M:%S',
                  time.localtime(block.timestamp))}")
            print(f"  Nonce: {block.nonce}")
            print(f"  交易数量: {len(block.transactions)}")

            for tx in block.transactions:
                print(f"    - {tx.sender[:8]}... -> {tx.recipient
                    [:8]}... : {tx.amount}")


# 使用示例
if __name__ == "__main__":
    # 创建区块链
    my_blockchain = Blockchain(difficulty=3)  # 使用较低难度以便
        快速演示

    # 添加一些交易
    my_blockchain.add_transaction(Transaction("Alice", "Bob", 50)
```

```
        )
my_blockchain.add_transaction(Transaction("Bob", "Charlie", 25))
    # 挖矿
    print("\n--- 开始挖矿 ---")
    my_blockchain.mine_pending_transactions("Miner1")

    # 添加更多交易
    my_blockchain.add_transaction(Transaction("Charlie", "Alice",
        10))
    my_blockchain.add_transaction(Transaction("Alice", "David",
        30))

    # 再次挖矿
    print("\n--- 开始挖矿 ---")
    my_blockchain.mine_pending_transactions("Miner2")

    # 打印区块链
    my_blockchain.print_chain()

    # 验证区块链
    print("\n" + "="*60)
    print(f"区块链有效性: {my_blockchain.is_chain_valid()}")

    # 查询余额
    print("\n账户余额:")
    print(f"  Miner1: {my_blockchain.get_balance('Miner1')}")
    print(f"  Miner2: {my_blockchain.get_balance('Miner2')}")
    print(f"  Alice: {my_blockchain.get_balance('Alice')}")
    print(f"  Bob: {my_blockchain.get_balance('Bob')}")
    print(f"  Charlie: {my_blockchain.get_balance('Charlie')}")
```

18.2 密码学算法实现

18.2.1 完整 SM3 实现

```
"""
SM3 Hash算法完整实现
基于国密标准GM/T 0004-2012
"""

import struct

# SM3初始值IV
IV = [
    0x7380166f, 0x4914b2b9, 0x172442d7, 0xda8a0600,
    0xa96f30bc, 0x163138aa, 0xe38dee4d, 0xb0fb0e4e
]

# SM3常量T
```

```
T = [0x79cc4519 if i < 16 else 0x7a879d8a for i in range(64)]
def left_rotate(x, n):    """循环左移"""
    return ((x << n) | (x >> (32 - n))) & 0xffffffff

def ff(x, y, z, j):
    """布尔函数FF"""
    if j < 16:
        return x ^ y ^ z
    else:
        return (x & y) | (x & z) | (y & z)

def gg(x, y, z, j):
    """布尔函数GG"""
    if j < 16:
        return x ^ y ^ z
    else:
        return (x & y) | (~x & z)

def p0(x):
    """置换函数P0"""
    return x ^ left_rotate(x, 9) ^ left_rotate(x, 17)

def p1(x):
    """置换函数P1"""
    return x ^ left_rotate(x, 15) ^ left_rotate(x, 23)

def sm3_compress(v, b):
    """SM3压缩函数"""
    # 消息扩展
    w = [0] * 68
    w_prime = [0] * 64

    # 将512位块分成16个32位字
    for i in range(16):
        w[i] = struct.unpack('>I', b[i*4:(i+1)*4])[0]

    # 消息扩展
    for i in range(16, 68):
        w[i] = p1(w[i-16] ^ w[i-9] ^ left_rotate(w[i-3], 15)) ^ \
               left_rotate(w[i-13], 7) ^ w[i-6]

    for i in range(64):
        w_prime[i] = w[i] ^ w[i+4]

    # 初始化寄存器
    a, b, c, d, e, f, g, h = v

    # 64轮压缩
    for j in range(64):
        ss1 = left_rotate((left_rotate(a, 12) + e + left_rotate(T
```

```
            [j], j)) & 0xffffffff, 7)
ss2 = ss1 ^ left_rotate(a, 12)        tt1 = (ff(a, b, c, j) + d +
         ss2 + w_prime[j]) & 0xffffffff
        tt2 = (gg(e, f, g, j) + h + ss1 + w[j]) & 0xffffffff

        d = c
        c = left_rotate(b, 9)
        b = a
        a = tt1
        h = g
        g = left_rotate(f, 19)
        f = e
        e = p0(tt2)

    # 输出
    return [
        (v[0] ^ a) & 0xffffffff,
        (v[1] ^ b) & 0xffffffff,
        (v[2] ^ c) & 0xffffffff,
        (v[3] ^ d) & 0xffffffff,
        (v[4] ^ e) & 0xffffffff,
        (v[5] ^ f) & 0xffffffff,
        (v[6] ^ g) & 0xffffffff,
        (v[7] ^ h) & 0xffffffff
    ]

def sm3_hash(message: bytes) -> str:
    """
    计算SM3 Hash值

    Args:
        message: 输入消息（bytes类型）

    Returns:
        64字符的十六进制字符串
    """
    # 消息填充
    msg_len = len(message) * 8  # 消息长度（位）

    # 添加'1'位（0x80 = 10000000）
    padded = message + b'\x80'

    # 添加'0'位，直到长度 ≡ 448 (mod 512)
    while (len(padded) * 8) % 512 != 448:
        padded += b'\x00'

    # 添加64位消息长度（大端序）
    padded += struct.pack('>Q', msg_len)

    # 初始化Hash值
```

```
    v = IV[:]
# 处理每个512位块    for i in range(0, len(padded), 64):
        block = padded[i:i+64]
        v = sm3_compress(v, block)

    # 转换为十六进制字符串
    return ''.join(f'{x:08x}' for x in v)


# 测试
if __name__ == "__main__":
    # 标准测试向量
    test_cases = [
        (b"abc", "66c7f0f462eeedd9d1f2d46b5e9d0b3e8e8e8
```

(a) 随机选择 k 个输入，至少有一对碰撞的概率

(b) 当 $n = 256$（SHA-256），$k = 2^{80}$ 时的碰撞概率

（提示：使用生日悖论公式 $p \approx 1 - e^{-k(k-1)/(2^{n+1})}$）

18.3.2 共识算法练习

1. **PoW** 难度计算：比特币当前 Bits 字段为 0x1705f3b8，计算：

   (a) 当前目标值

   (b) 当前难度（相对于最大目标值）

   (c) 预计每秒需要进行多少次 Hash 计算才能找到一个有效区块（假设 10 分钟出块）

2. **PBFT** 投票分析：在一个 7 节点的 PBFT 系统中：

   (a) 最多可以容忍多少个恶意节点？

   (b) 每个阶段需要收到多少张投票才能推进？

   (c) 如果第 3 个节点是主节点且是恶意的，系统还能正常工作吗？

3. **PoS** 质押收益：以太坊 2.0 中，假设：

   - 全网质押总量：1000 万 ETH
   - 你的质押：32 ETH
   - 在线率：98%
   - 基础奖励因子：64

   计算你的年化收益率和年收益（ETH）。

18.3.3 编程练习

1. 实现简化版 **Merkle Tree**：编写一个 Merkle Tree 类，支持：

   - 从交易列表构建 Merkle Tree
   - 计算 Merkle Root
   - 生成 Merkle Proof
   - 验证 Merkle Proof

2. 实现 **UTXO** 追踪：编写一个 UTXO 管理器，支持：

   - 添加新的 UTXO
   - 花费 UTXO（标记为已花费）
   - 查询某个地址的所有未花费 UTXO
   - 计算地址余额

3. 实现简单 **PoW** 挖矿：编写一个挖矿程序，输入区块头和难度，输出满足条件的 Nonce 和 Hash。

内部资料

18.3.4 综合思考题

1. 安全性分析：为什么比特币使用 SHA-256 两次（SHA-256d）？一次不够吗？
2. 扩展性讨论：比特币的 10 分钟出块时间和 1MB 区块限制是如何影响其扩展性的？有哪些改进方案？
3. 隐私性分析：比特币的 UTXO 模型提供了什么程度的隐私保护？有哪些方法可以追踪比特币交易？
4. 能源消耗：比较 PoW 和 PoS 的能源消耗。如果比特币全网算力为 200 EH/s，估算年耗电量。
5. 量子计算威胁：如果大规模量子计算机出现，哪些区块链密码学会受到影响？哪些不会？为什么？

内部资料